

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN, ĐHQG-HCM

KHOA MẠNG MÁY TÍNH VÀ TRUYỀN THÔNG



BÁO CÁO ĐỒ ÁN MÔN HỌC

**ĐỀ TÀI: XÂY DỰNG ỨNG DỤNG TRÒ CHƠI ĐUA XE PIXEL
DRIFT**

Môn học: NT106.Q12 - Lập trình mạng căn bản

Thực hiện bởi nhóm 04, bao gồm:

1.	Nguyễn Duy Hưng	24520602	Trưởng nhóm
2.	Ngô Xuân Minh Hoàng	24520549	Thành viên
3.	Nguyễn Hoàng Khánh Đăng	24520254	Thành viên
4.	Lê Tiến Hưng	24520596	Thành Viên
5.	Hoàng Phi Hùng	24520587	Thành viên

Link github: <https://github.com/HugndUIT/Pixel-Drift-NT106>

Thông tin liên lạc: 24520602@gm.uit.edu.vn

Giảng viên hướng dẫn: Lê Minh Khánh Hội

Thời gian thực hiện: 08/09/2025 - 10/12/2025

MỤC LỤC

MỤC LỤC	2
LỜI CẢM ƠN	4
Chương I: TỔNG QUAN.....	5
1. Lý do chọn đề tài.....	5
2. Mục tiêu nghiên cứu	5
3. Đối tượng nghiên cứu	5
3.1. Đối tượng.....	5
3.2. Phạm vi.....	6
4. Khảo sát.....	6
5. Nội dung, phương pháp thực hiện đề tài.....	6
6. Phân chia công việc.....	7
Chương II: CƠ SỞ LÝ THUYẾT	9
1. Tổng quan về công nghệ sử dụng	9
2. Kỹ thuật đồng bộ dữ liệu thời gian thực (Real-time Synchronization)	9
2.1. Tại sao sử dụng Socket thay vì Database Realtime:	9
2.2. Áp dụng vào trò chơi Pixel Drift:.....	9
3. Hệ quản trị cơ sở dữ liệu SQLite:	10
3.1. Tại sao nên sử dụng SQLite:	10
3.2. Áp dụng SQLite vào xây dựng game:	10
Chương III: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG.....	11
1. Kiến trúc hệ thống.....	11
2. Network stack và packet structure	12
2.1. Network stack.....	12
2.2. Packet structure	13
3. User story và flow hoạt động	17
4. Use case.....	30
4.1. Tổng quan.....	30
4.2. Tác nhân (Actor):	30
4.3. Chi tiết các nhóm chức năng	30
5. Database	33
5.1. Tổng quan.....	33
5.2. Chi tiết các bảng (Tables).....	33
5.3. Mối quan hệ giữa các bảng (Relationship Analysis).....	34
Chương IV: XÂY DỰNG ỨNG DỤNG.....	35
1. Tương quan giữa mục tiêu lý thuyết và thực tế	35
2. Các giao diện chính của ứng dụng	35

3. Kết quả đề tài	42
Chương V: KIỂM TRA ỨNG DỤNG	43
1. Kiểm tra chức năng sản phẩm.....	43
2. Khả năng chịu tải	49
3. Độ trễ.....	49
4. Các tác động môi trường.....	49
5. Kết luận	49
Chương VI: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	49
1. Kết luận	49
2. Ưu điểm.....	50
3. Nhược điểm.....	50
4. Hướng phát triển	50
TÀI LIỆU THAM KHẢO.....	51

LỜI CẢM ƠN

Trước tiên, với tất cả lòng biết ơn, chúng em xin gửi lời cảm ơn trân trọng và sâu sắc đến cô Lê Minh Khánh Hội và thầy Trần Văn Như Ý - người cô và người thầy hướng dẫn đã hết sức nhiệt tình hỗ trợ và giúp đỡ nhóm em trong suốt quá trình thực hiện đồ án môn học này. Những định hướng, bổ sung, góp ý của thầy, cô là kim chỉ nam và nền tảng cơ sở góp phần giúp chúng em có những bước đi đúng đắn, đạt được kết quả tốt nhất trong việc xây dựng, phát triển ứng dụng trò chơi và hoàn thiện đồ án môn học.

Mặc dù nhóm đã rất cố gắng nhưng kiến thức và kinh nghiệm có hạn, và cũng là lần đầu tiên nhóm em được tự tay hoàn thiện đồ án nên sản phẩm của em còn có nhiều thiếu sót. Nhóm em rất mong nhận được sự thông cảm và đóng góp ý kiến của thầy, cô.

Một lần nữa, chúng em xin chân thành cảm ơn và xin gửi lời chúc sức khỏe đến cô Lê Minh Khánh Hội và thầy Trần Văn Như Ý. Kính chúc thầy, cô luôn luôn tiến tới và thành công trong sự nghiệp theo đuổi tri thức của mình.

TP. Hồ Chí Minh, ngày 15 tháng 12 năm 2025

Chương I: TỔNG QUAN

1. Lý do chọn đề tài

Trong bối cảnh áp lực học tập và làm việc ngày càng cao, nhu cầu giải trí thông qua trò chơi điện tử đã trở thành một phần không thể thiếu trong đời sống hiện đại. Tuy nhiên, dưới góc độ của sinh viên Công nghệ thông tin, game không đơn thuần là công cụ giải trí mà còn là một hệ thống phức tạp, đòi hỏi sự kết hợp chặt chẽ giữa tư duy logic, kỹ thuật đồ họa và đặc biệt là kỹ thuật lập trình mạng.

Hiện nay, xu hướng người chơi đang chuyển dịch mạnh mẽ từ các trò chơi đơn thi đấu với máy (AI) sang các tựa game đối kháng trực tuyến để tìm kiếm sự cạnh tranh. Nếu như các ứng dụng quản lý truyền thống thường mang tính tĩnh, thì một tựa game đua xe lại yêu cầu khả năng xử lý thời gian thực với độ chính xác đến từng mili-giây.

Chính vì những thách thức kỹ thuật thú vị đó, nhóm em quyết định chọn đề tài **“Xây dựng phần mềm trò chơi đua xe - Pixel Drift”**. Đây là cơ hội để nhóm kiểm chứng và áp dụng tổng hợp các kiến thức cốt lõi đã học: lập trình mạng, xây dựng giao diện cho đến quản lý dữ liệu.

2. Mục tiêu nghiên cứu

- Xây dựng phần mềm trò chơi có giao diện trực quan, dễ cài đặt và sử dụng, đảm bảo tính ổn định khi hoạt động trên môi trường mạng LAN.
- Hỗ trợ kết nối và tương tác giữa 2 người chơi trong thời gian thực và đảm bảo cho độ trễ truyền tin được ở mức thấp nhất.
- Hỗ trợ quản lý, cập nhật và lưu trữ thông tin người chơi (tài khoản, điểm số) một cách chính xác và bảo mật nhất.

3. Đối tượng nghiên cứu

3.1. Đối tượng

- Nghiên cứu kỹ thuật lập trình Socket, sử dụng giao thức mạng TCP/IP thiết lập kết nối để truyền tải các gói tin (toạ độ, trạng thái) giữa hai người chơi trong mạng LAN.

- Tìm hiểu và vận dụng ngôn ngữ C#, nền tảng Windows Forms .NET và thư viện đồ hoạ GDI+ để xây dựng giao diện, xử lý logic cho trò chơi.
- Tìm hiểu về hệ quản trị cơ sở dữ liệu nhúng SQLite để thực hiện lưu trữ thông tin người chơi như tài khoản, mật khẩu và bảng xếp hạng điểm số

3.2. Phạm vi

- Đề tài được xây dựng cho máy tính, laptop chạy hệ điều hành windows. Đề tài được thực hiện, hoàn thiện một sản phẩm cụ thể và được kiểm tra, thử nghiệm ở quy mô nhỏ.

4. Khảo sát

Hiện nay trên thị trường đã có rất nhiều các loại game đua xe nổi tiếng (Asphalt 9: Legends, Racing Fever, Traffic Rider,...) và những game này còn được đầu tư xây dựng đồ hoạ 3D rất đẹp mắt cùng với các công cụ làm game (Game Engine) hỗ trợ việc làm game mà không cần code nhiều. Vì vậy, mục tiêu của chúng em khi chọn đề án này không phải là cải tiến hay phát triển ứng dụng để cạnh tranh với các tựa game trên, mà là để rèn luyện, vận dụng các kiến thức được học trong môn “Lập trình mạng căn bản” vào thực tiễn.

5. Nội dung, phương pháp thực hiện đề tài

Nội dung 1: Phân tích quy trình hoạt động và thiết kế giao diện người dùng.

- **Mục đích:** Xác định rõ luồng đi của trò chơi (Game Flow), thiết kế các màn hình tương tác trực quan và chuẩn bị cơ sở dữ liệu để lưu trữ thông tin.
- **Phương pháp:**
 - + Phân tích các yêu cầu chức năng để xác định các thành phần cần thiết trên giao diện (ví dụ: cần xe đua, cần đồng hồ đếm giờ).
 - + Sử dụng kiến thức về Lập trình hướng sự kiện (Event-Driven Programming) để bắt các sự kiện từ bàn phím (người chơi ấn nút) và sự kiện từ hệ thống (Timer đếm giờ).
 - + Sử dụng các công cụ có sẵn trong Visual Studio Toolbox như PictureBox, Panel, Timer để xây dựng bố cục trò chơi.
 - + Thiết kế giao diện theo tư duy trải nghiệm người dùng (UX), đảm bảo tính thẩm mỹ và dễ sử dụng.

- Kết quả thực hiện:

- + Xây dựng hoàn thiện lưu đồ thuật toán (Flowchart) mô tả luồng xử lý của trò chơi.
- + Hoàn thành các giao diện (Form): Đăng ký, Đăng nhập, Quên mật khẩu, Đổi mật khẩu, Bảng thông tin, Lobby, Màn hình chơi game (Gameplay) và Bảng xếp hạng.
- + Xây dựng cơ chế điều khiển xe và xử lý va chạm trên giao diện WinForms.

Nội dung 2: Nghiên cứu công nghệ và hiện thực hóa các chức năng mạng.

- **Mục đích:** Nắm vững và áp dụng thành công các công nghệ lõi: lập trình mạng (Socket TCP/IP), xử lý đồ họa (GDI+, Guna) và thao tác dữ liệu (SQLite) để game hoạt động trơn tru.
- **Phương pháp:**
 - + Tìm hiểu tài liệu kỹ thuật về System.Net.Sockets để xử lý gửi/nhận gói tin giữa hai máy tính.
 - + Nghiên cứu thư viện System.Drawing (GDI+, Guna.UI2) để vẽ và xử lý chuyển động, va chạm của xe trên màn hình.
 - + Tham khảo các tài liệu chuẩn của Microsoft và cộng đồng lập trình viên về cách tối ưu hóa hiệu năng khi chạy đa luồng (Multi-threading).
- **Kết quả thực hiện:**
 - + Nắm vững quy trình bắt tay 3 bước của TCP/IP.
 - + Hiện thực hóa được Server (quản lý kết nối) và Client (xử lý hiển thị).
 - + Kết nối thành công Database để thực hiện các chức năng Thêm/Lưu/Tra cứu điểm số.

6. Phân chia công việc

Thành viên	Nhiệm vụ
Nguyễn Duy Hưng (20%)	- Code tính năng đăng ký. - Code để server và phòng game có thể kết nối với nhau và gửi gói tin qua lại. - Code tính năng quản lý người chơi ở phía client. - Fix lỗi. - Viết báo cáo.
Ngô Xuân Minh Hoàng (20%)	- Code tính năng mã hoá mật khẩu ở giao diện đăng ký và đăng nhập. - Code tính năng để client tự động tìm đến IP server. - Code logic game và gửi gói tin yêu cầu cập nhật trạng thái đến server. - Fix lỗi. - Viết báo cáo.
Nguyễn Hoàng Khánh Đăng (20%)	- Code tính năng quản lý người chơi ở phía client. - Code server xử lý các gói tin bên trong phòng chơi game. - Thêm tính năng tính điểm. - Thêm tính năng hiển thị thông tin người chơi. - Fix lỗi. - Phụ viết báo cáo và làm slide
Lê Tiến Hưng (20%)	- Code tính năng xem bảng điểm. - Căn chỉnh giao diện game đẹp hơn. - Thực hiện truy vấn database để lấy dữ liệu. - Code tính năng chống tài khoản đăng nhập cùng lúc nhiều nơi. - Fix lỗi. - Làm slide
Hoàng Phi Hùng (20%)	- Code tính năng đổi mật khẩu, quên mật khẩu và đăng nhập. - Code lobby. - Thêm tính năng xử lý các gói tin tạo phòng và vào phòng ở phía server. - Fix lỗi. - Làm slide

Chương II: CƠ SỞ LÝ THUYẾT

1. Tổng quan về công nghệ sử dụng

- Trò chơi “Pixel Drift” được phát triển dựa trên nền tảng Microsoft .NET Framework.
 - **Phía client và phía server:** Cả hai được xây dựng thống nhất bằng C# trên nền tảng Windows Forms để đảm bảo tính tương thích cao nhất.
 - **Giao diện:** Giao diện được thiết kế bằng thư viện Guna.UI2.WinForms và kết hợp với công cụ đồ họa gốc GDI+ để tạo hiệu ứng mượt mà nhưng vẫn đảm bảo tính thẩm mỹ, đẹp mắt.
 - **Kết nối mạng:** Khác với các phần mềm ứng dụng quản lý sử dụng API web trung gian, hệ thống game sử dụng giao thức Socket TCP/IP để truyền tải dữ liệu trực tiếp, đảm bảo tốc độ phản hồi tức thì (Real-time) cho game đua xe.
 - **Lưu trữ dữ liệu:** Dữ liệu người dùng và bảng xếp hạng được quản lý bởi SQLite, một hệ quản trị cơ sở dữ liệu nhỏ gọn, tích hợp sẵn bên trong ứng dụng mà không cần cài đặt máy chủ phụ thuộc.

2. Kỹ thuật đồng bộ dữ liệu thời gian thực (Real-time Synchronization)

2.1. Tại sao sử dụng Socket thay vì Database Realtime:

Đối với game tốc độ cao như “Pixel Drift”, việc đọc/ghi liên tục vào Database sẽ gây ra độ trễ lớn. Thay vào đó, nhóm em sẽ sử dụng kỹ thuật truyền gói tin trực tiếp qua Socket:

- **Tốc độ nhanh:** Các gói tin JSON chứa tọa độ (X, Y) và trạng thái xe được gửi đi liên tục (60-80 lần/giây), giúp chuyển động của đối thủ trên màn hình người chơi trở nên mượt mà.
- **Luồng dữ liệu liên tục:** Kết nối giữa Client và Server luôn được duy trì (Keep-Alive), không mất thời gian thiết lập lại kết nối cho mỗi lần gửi tin.

2.2. Áp dụng vào trò chơi Pixel Drift:

Trong đề tài này, Socket được dùng để đồng bộ toàn bộ trạng thái game:

- **Game State:** Đồng bộ vị trí của xe và các chương ngại vật giữa màn hình của 2 người chơi.

- **Sự kiện:** Gửi các tín hiệu va chạm và xử lý các tín hiệu đẩy ngay lập tức.

3. Hệ quản trị cơ sở dữ liệu SQLite:

3.1. Tại sao nên sử dụng SQLite:

SQLite là một thư viện C thực hiện một công cụ cơ sở dữ liệu SQL khép kín, không cần máy chủ và không cần cấu hình.

- **Tính di động:** Toàn bộ dữ liệu nằm gọn trong một file duy nhất (.sqlite). Người dùng chỉ cần copy thư mục game là có thể chạy ngay mà không cần cài đặt SQL Server hay MySQL.
- **Hiệu suất:** Tốc độ truy vấn cực bộ cực nhanh, phù hợp để lưu trữ thông tin đăng nhập và bảng điểm cá nhân.

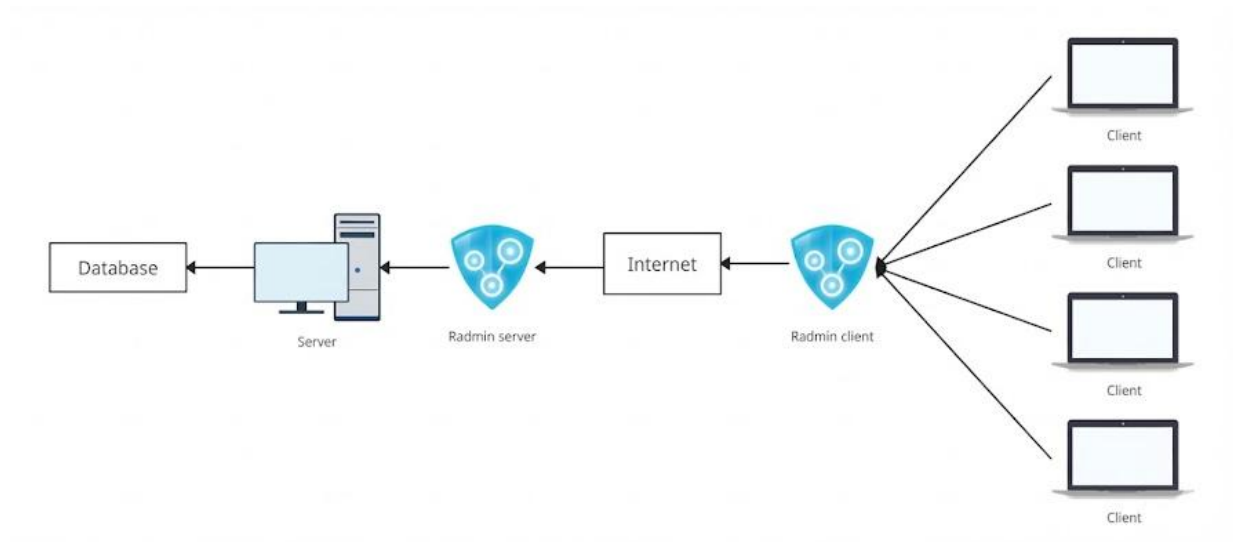
3.2. Áp dụng SQLite vào xây dựng game:

Trong dự án này, SQLite đóng vai trò là kho lưu trữ an toàn và bảo mật:

- **Bảng Info_User:** Lưu trữ thông tin đăng ký (Tên đăng nhập, Email, Ngày sinh) và Mật khẩu (đã được mã hóa SHA-256).
- **Bảng Scores:** Lưu trữ lịch sử đấu, số trận thắng, và tổng điểm tích lũy để hiển thị lên Bảng xếp hạng (Leaderboard).

Chương III: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

1. Kiến trúc hệ thống



Sơ đồ thể hiện rõ 4 thành phần chính của hệ thống :

Client (Máy khách): Là các máy tính cá nhân chạy ứng dụng game Pixel Drift, được phát triển trên nền tảng Windows Forms (.NET) sử dụng ngôn ngữ C#. Tại đây, ứng dụng chịu trách nhiệm thu thập các tín hiệu ví dụ: điều khiển từ bàn phím, thông tin đăng nhập, đăng ký,... Sau đó gửi dữ liệu cho server, sau khi server phản hồi sẽ hiển thị lên UI. Quá trình giao tiếp với Server sử dụng kết nối socket thuần với giao thức TCP/IP tại cổng 1111, và toàn bộ gói tin được đóng gói theo định dạng JSON để đảm bảo tốc độ và tính linh hoạt.

Hạ tầng mạng (Network): Hệ thống vận hành trên nền tảng Internet nhưng được bảo vệ thông qua lớp mạng trung gian là phần mềm Radmin VPN. Giải pháp này thiết lập một mạng riêng ảo (VPN), tạo ra các đường hầm kết nối được mã hóa giữa các máy Client và Server. Cơ chế này cho phép ứng dụng hoạt động ổn định như trong môi trường mạng nội bộ (LAN) với các địa chỉ IP ảo, giải quyết vấn đề kết nối từ xa mà không cần cấu hình IP công cộng (Public IP) phức tạp.

Server (Máy Chủ): Đây là thành phần trung tâm được xây dựng bằng ngôn ngữ C#, hoạt động theo mô hình Authoritative Server (Máy chủ toàn quyền). Server sử dụng giao thức TCP để lắng nghe và xử lý đa luồng (Multi-threading) cho nhiều người chơi cùng lúc. Bên trong, Server quản lý các phòng game, thực hiện toàn bộ các tác vụ nặng như: xử lý logic trò chơi, tính toán vật lý va chạm, đồng bộ tọa độ thời gian thực và là cổng duy nhất kiểm soát luồng dữ liệu ra vào Database.

Cơ sở dữ liệu (Database): Hệ thống sử dụng SQLite để quản lý cơ sở dữ liệu. Dữ liệu được lưu trữ cục bộ trên máy chủ. Thành phần này đảm bảo khả năng lưu trữ bền vững cho thông tin tài khoản đăng nhập và bảng xếp hạng thành tích người chơi. Việc đặt cơ sở dữ liệu phía sau Server giúp bảo mật tuyệt đối, ngăn chặn các truy cập trái phép trực tiếp từ phía Client.

2. Network stack và packet structure

2.1. Network stack

Tầng (Layer)	Giao thức (Protocol)
4. Structured Information	JSON Serialization
3. Message	Custom JSON
2. Stream	TCP
1. Packet	IP

4. Structured Information (Thông tin có cấu trúc)

- **Chức năng:** Tầng này định dạng dữ liệu có ý nghĩa cho ứng dụng.
- **Trong game:** Mọi thông điệp được gửi giữa Client và Server đều là các đối tượng C# vô danh được tuần tự hóa thành chuỗi JSON

3. Message (Thông điệp)

- **Chức năng:** Tầng này định nghĩa các quy tắc giao tiếp.
- **Trong game (Custom JSON Protocol):** Bên gửi phải có trường action và bên phản hồi phải có trường status.

2. Stream (Luồng)

- **Chức năng:** Tầng này cung cấp kết nối logic, đáng tin cậy giữa hai điểm cuối.
- **Trong game:** TCP (Transmission Control Protocol) được sử dụng. Client sử dụng TcpClient để kết nối đến Server. Server sử dụng

TcpListener để chấp nhận kết nối. Dữ liệu được gửi và nhận thông qua NetworkStream.

1. Packet (Gói)

- **Chức năng:** Tầng này chịu trách nhiệm định tuyến các gói dữ liệu qua mạng vật lý.
- **Trong game:** IP (Internet Protocol) được sử dụng. Địa chỉ IP được sử dụng để xác định Server và Client. Client có thể dùng UDP để khám phá địa chỉ IP của Server trước khi thiết lập kết nối TCP.

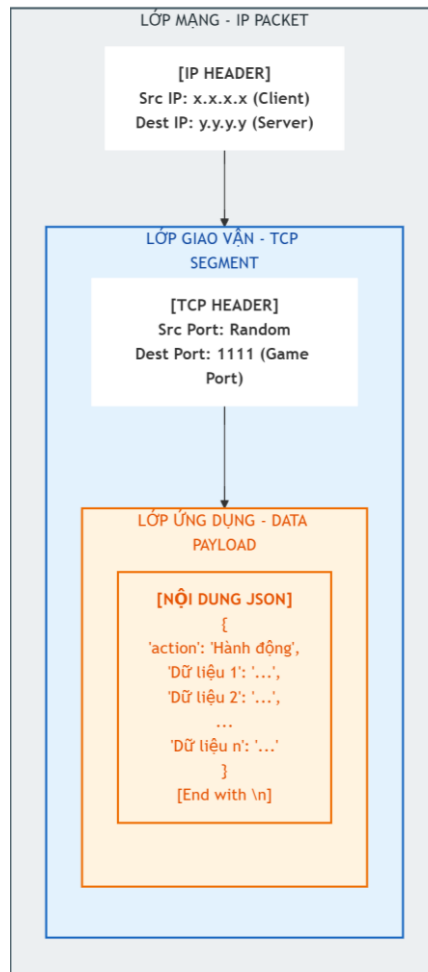
2.2. Packet structure

- Cấu trúc chung: Gói tin có phần payload được quy định như sau: request sẽ phải có trường action, gói tin response sẽ phải có trường status. Sau đó đóng gói thêm dần các header gồm Port và IP.

- Định dạng json

```
1 {                                     1 {
2   "action": "Hành động"             2   "status": "Trạng thái"
3   "Dữ liệu 1": "...                 3   "Dữ liệu 1": "...
4   "Dữ liệu 2": "...                 4   "Dữ liệu 2": "...
5   ...                               5   ...
6   "Dữ liệu n": "...                 6   "Dữ liệu n": "...
7 }                                     7 }
```

- Định dạng gói tin sau khi đóng gói



- Chi tiết tất cả các nội dung json và chức năng:
 - Client → Server:
 - Login Request: {"action": "login", "username": "...", "password": "..."} → Yêu cầu đăng nhập với tên và mật khẩu (đã mã hóa SHA256).
 - Register Request: {"action": "register", "email": "...", "username": "...", "password": "...", "birthday": "..."} → Gửi thông tin đăng ký tài khoản mới.
 - Get Info Request: {"action": "get_info", "username": "..."} → Lấy thông tin chi tiết (Email, ngày sinh) của người dùng để hiển thị.

- Forgot Password Request: {"action": "forgot_password", "email": "..."} → Yêu cầu gửi mã token reset mật khẩu qua email.
- Change Password Request: {"action": "change_password", "email": "...", "token": "...", "new_password": "..."} → Gửi mật khẩu mới kèm token xác thực để đổi mật khẩu.
- Create Room Request: {"action": "create_room", "username": "..."} → Yêu cầu Server tạo một phòng chơi mới. Server sẽ sinh Room ID ngẫu nhiên.
- Join Room Request: {"action": "join_room", "room_id": "...", "username": "..."} → Yêu cầu tham gia vào một phòng đã có sẵn bằng ID.
- Set Ready Request: {"action": "set_ready", "ready_status": "true"} → Báo cho Server biết người chơi đã sẵn sàng để bắt đầu.
- Input Move: {"action": "move", "player": 1, "direction": "left", "state": "down"} → Gửi lệnh di chuyển khi nhấn/thả phím. state là trạng thái phím (down/up).
- Add Score: {"action": "add_score", "win_count": 1, "total_score": 1000, ...} → Gửi kết quả trận đấu về Server để lưu vào Database.
- Get Top: {"action": "get_scoreboard", "top_count": 50} → Yêu cầu lấy danh sách Top 50 người chơi xuất sắc nhất.
- Search: {"action": "search_player", "search_text": "Hung"} → Tìm kiếm lịch sử đấu của một người chơi cụ thể.
- Data Result: {"action": "scoreboard_data", "data": "..."} → Trả về dữ liệu bảng xếp hạng dưới dạng chuỗi văn bản.
- Server → Client:
 - Login/Register/Get Info/Forgot Password/Change Password Response: {"status": "success" / "error", "message": "..."} →

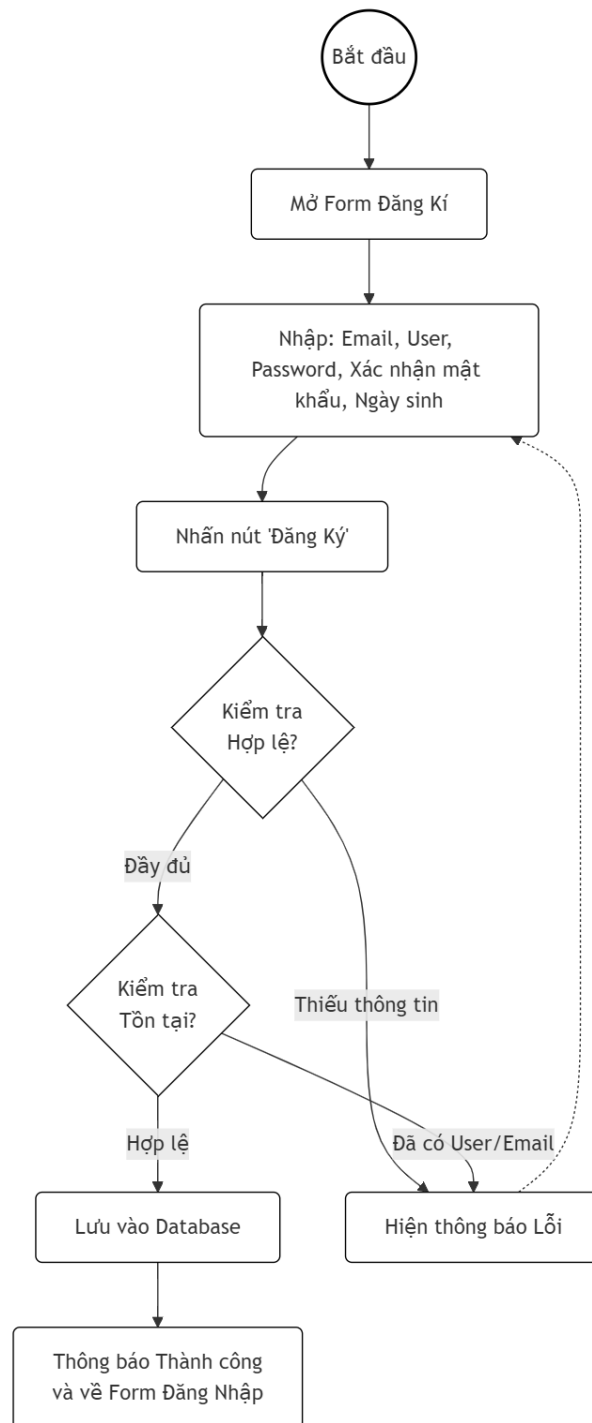
Thông báo trạng thái thành công hay lỗi của các request đăng nhập, đăng ký, lấy thông tin, đổi mật khẩu, quên mật khẩu.

- Force Logout Response: {"status": "force_logout", "message": "..."} → Thông báo ép buộc đăng xuất (Kick) khi tài khoản đăng nhập ở nơi khác.
- Create Room Response: {"status": "create_room_success", "room_id": "...", "player_number": 1} → Xác nhận tạo phòng thành công và trả về ID phòng để hiển thị.
- Join Room Response: {"status": "join_room_success", "room_id": "...", "player_number": 2} → Xác nhận vào phòng thành công.
- Update Ready: {"action": "update_ready_status", "player1_ready": true, ...} → Cập nhật trạng thái sẵn sàng của cả 2 người chơi lên giao diện.
- Countdown: {"action": "countdown", "time": 5} → Đếm ngược 5 giây trước khi bắt đầu game.
- Start Game: {"action": "start_game"} → Lệnh bắt đầu trò chơi chính thức. Client sẽ ấn nút Ready và hiện xe.
- Game State: {"action": "update_game_state", "ptb_player1": {"X":...}, ...} → **Gói tin nặng nhất**. Chứa tọa độ X,Y của tất cả vật thể (xe, đường, chướng ngại vật) để Client vẽ lại.
- Sync Time: {"action": "update_time", "time": 59} → Đồng bộ thời gian còn lại của ván đấu (60s đếm ngược).
- Sync Score: {"action": "update_score", "p1_score": 100, "p2_score": 200} → Cập nhật điểm số thời gian thực dựa trên quãng đường đi được.
- Play Sound: {"action": "play_sound", "sound": "buff/debuff/hit_car"} → Yêu cầu Client phát âm thanh hiệu ứng khi ăn buff hoặc va chạm.

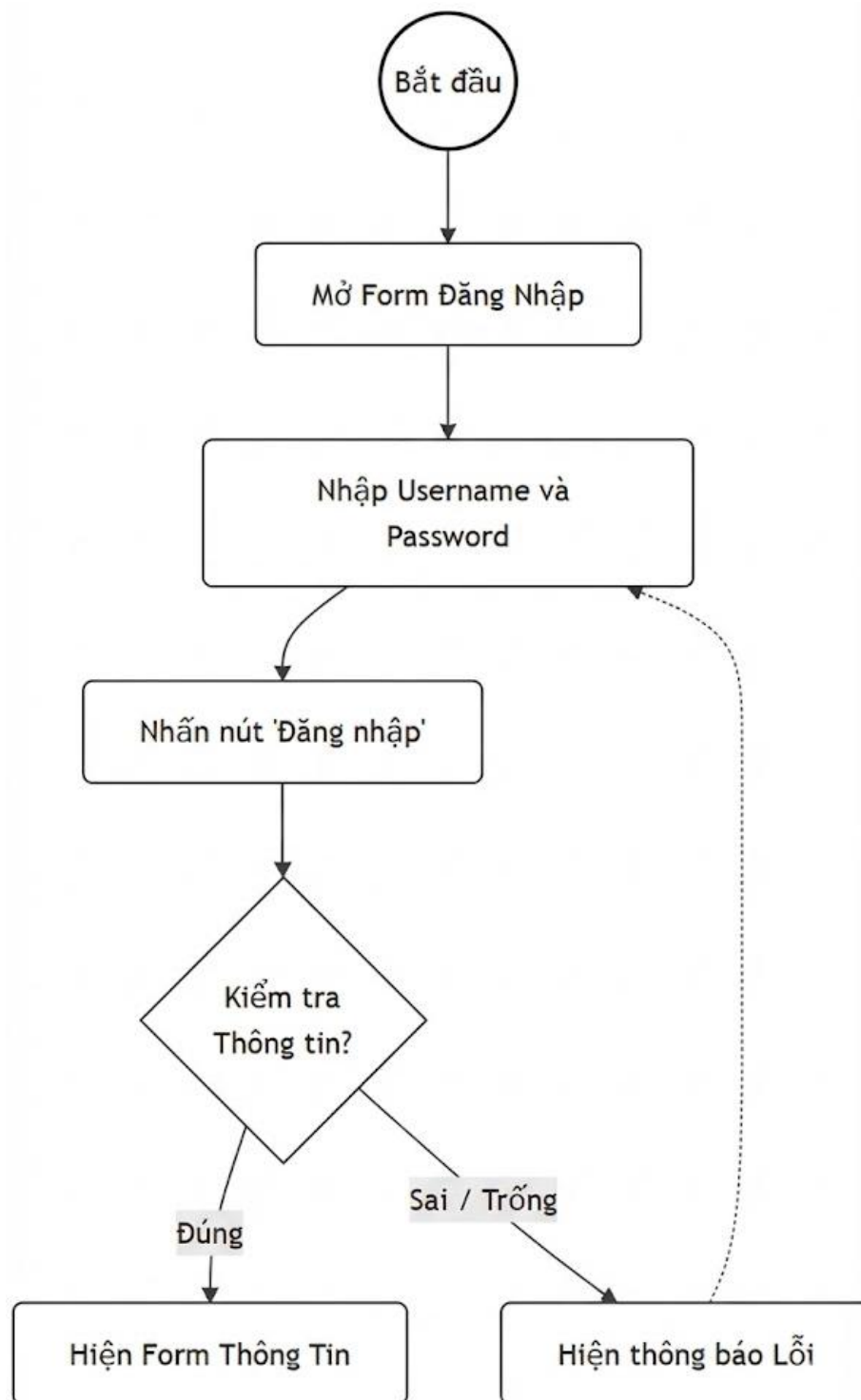
- Game Over: {"action": "game_over"} → Thông báo kết thúc trò chơi khi hết giờ.
- Leave Room: {"action": "leave_room"} → Thông báo người chơi thoát phòng (hoặc tắt game).
- Opponent Left: {"action": "player_disconnected", "name": "..."}
→ Báo cho người còn lại biết đối thủ đã thoát để dừng game.

3. User story và flow hoạt động

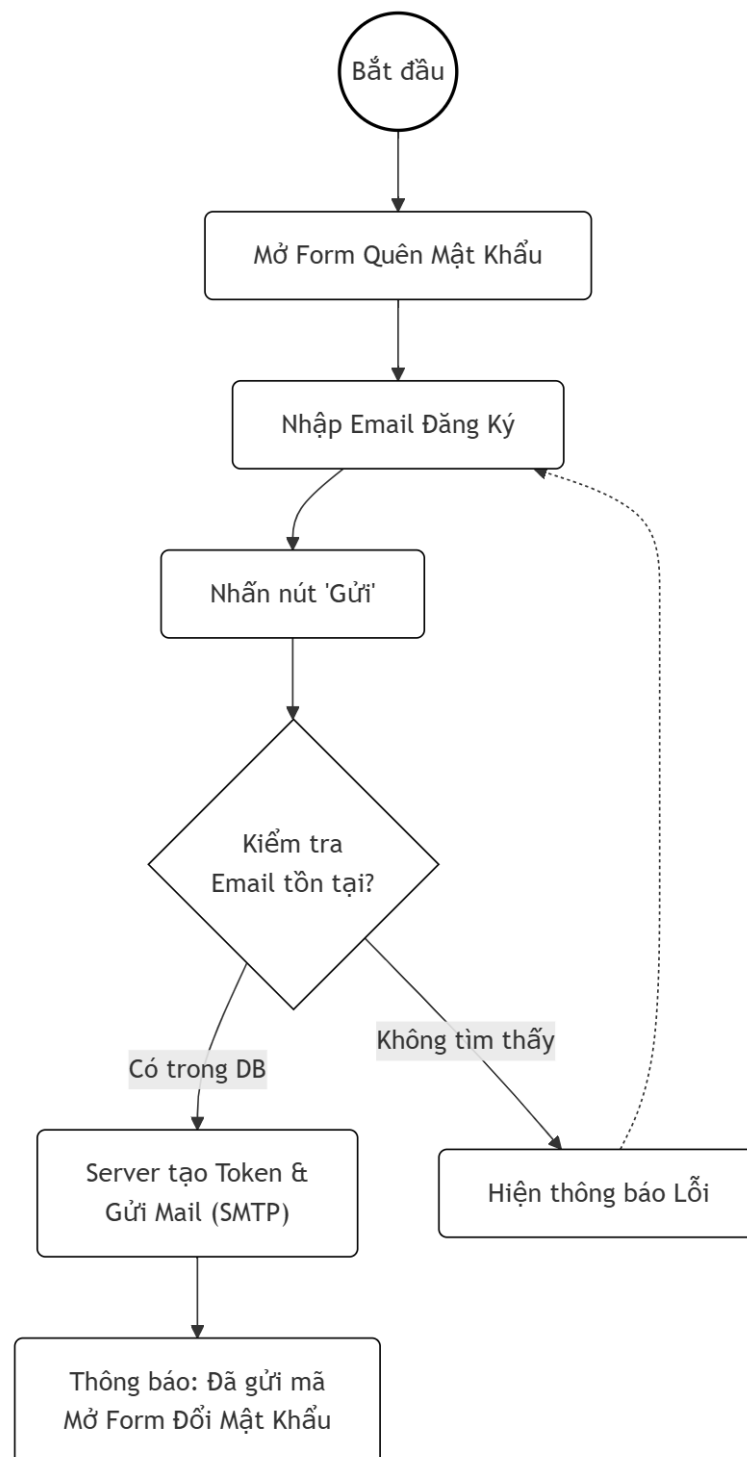
- Đăng ký: Là một người chơi mới, tôi muốn đăng ký tài khoản với tên, email và ngày sinh để có thể bắt đầu tham gia trò chơi.
 - Flow hoạt động đăng ký:



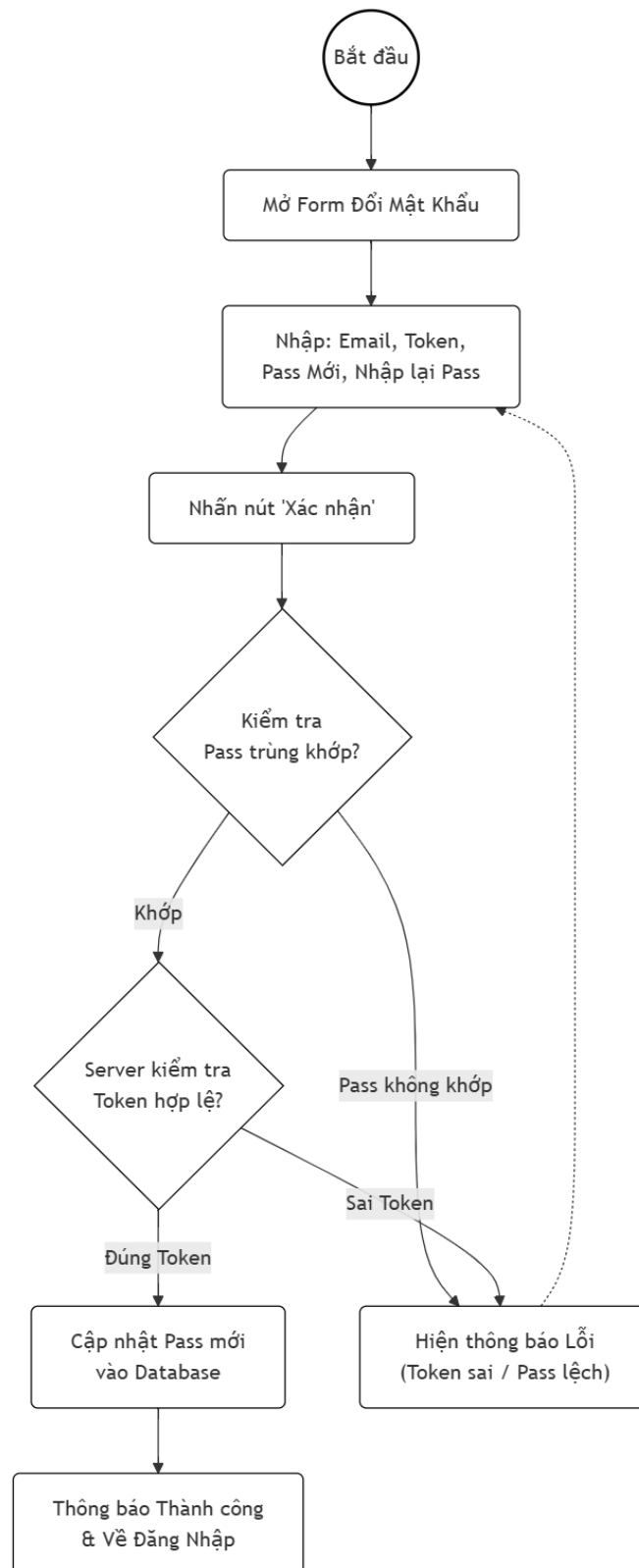
- Đăng nhập: Là một người chơi đã đăng ký, tôi muốn đăng nhập vào hệ thống bằng tài khoản của mình để truy cập vào sảnh chờ.
 - Flow hoạt động đăng nhập:



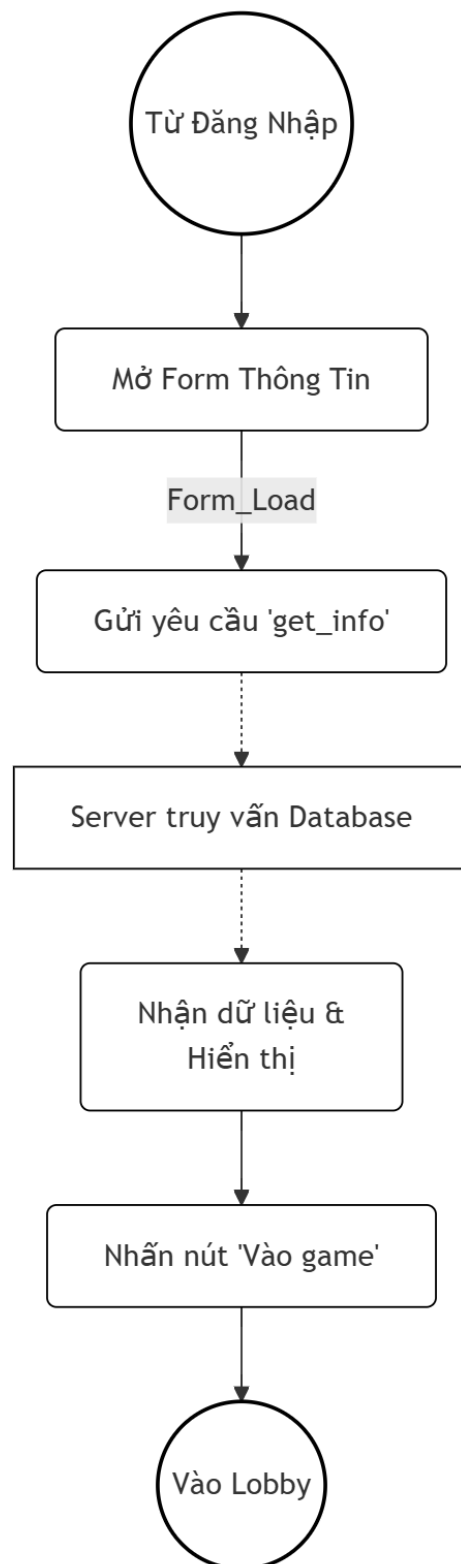
- Quên mật khẩu: Là một người chơi quên mật khẩu, tôi muốn yêu cầu cấp lại mật khẩu qua email để khôi phục quyền truy cập tài khoản.
 - Flow hoạt động quên mật khẩu:



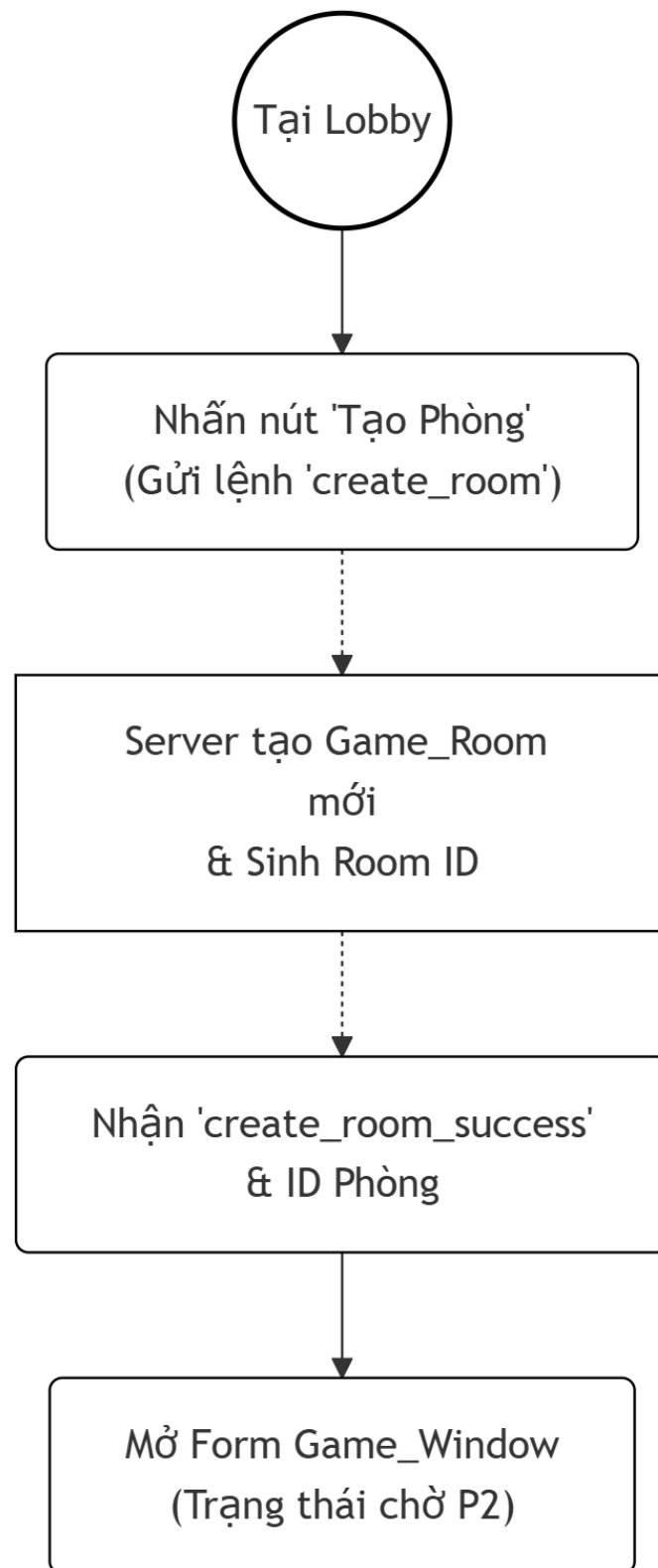
- **Đổi mật khẩu:** Là một người chơi, tôi muốn nhập token và mật khẩu mới để thay đổi mật khẩu cũ nhằm bảo mật tài khoản.
 - Flow hoạt động đổi mật khẩu:



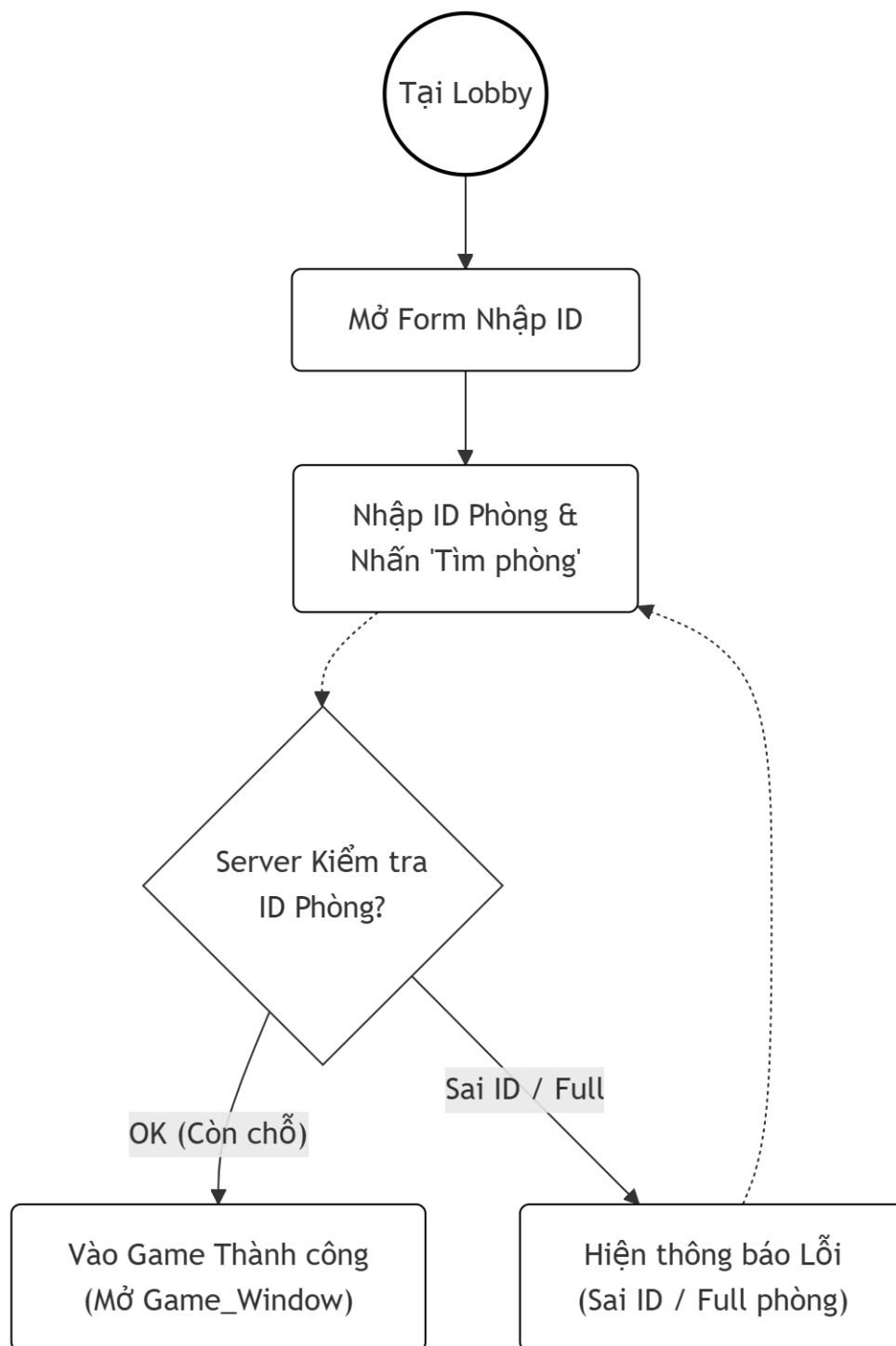
- Xem thông tin: Là một người chơi, tôi muốn xem thông tin cá nhân (email, ngày sinh) để kiểm tra độ chính xác của tài khoản.
 - Flow hoạt động xem thông tin:



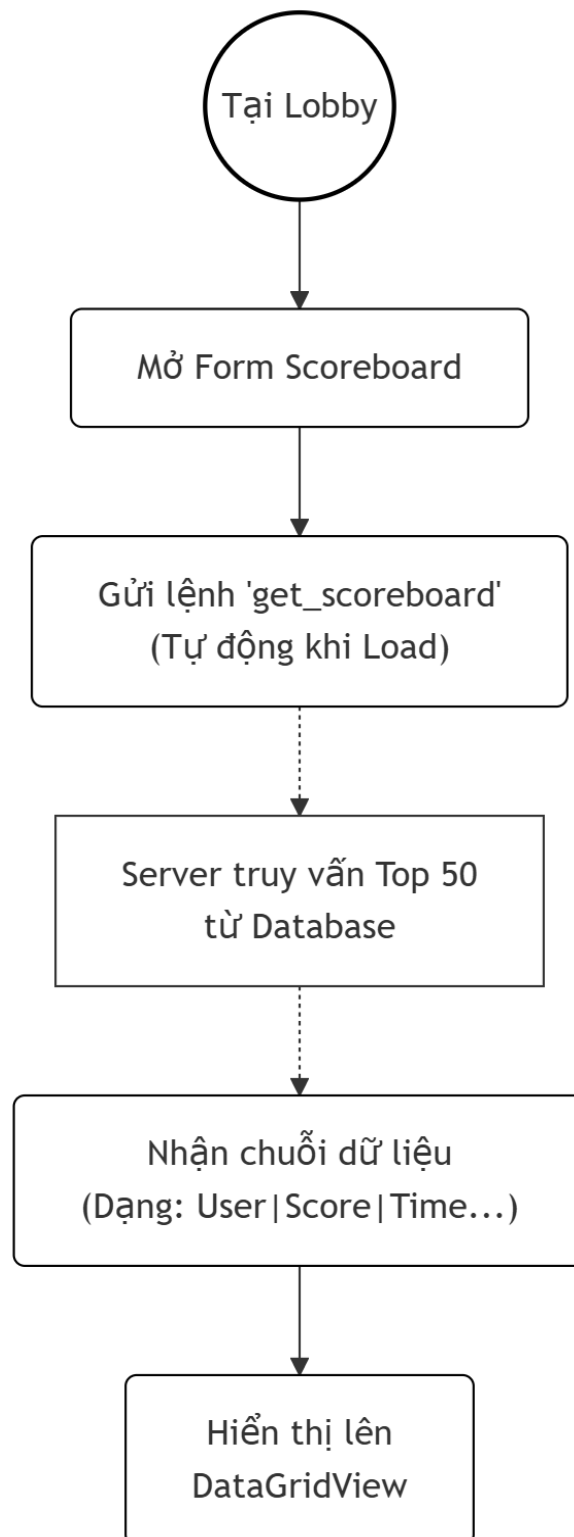
- Tạo phòng: Là một người chơi muốn làm chủ phòng, tôi muốn tạo một phòng chơi mới để đợi người khác vào thách đấu.
 - Flow hoạt động tạo phòng



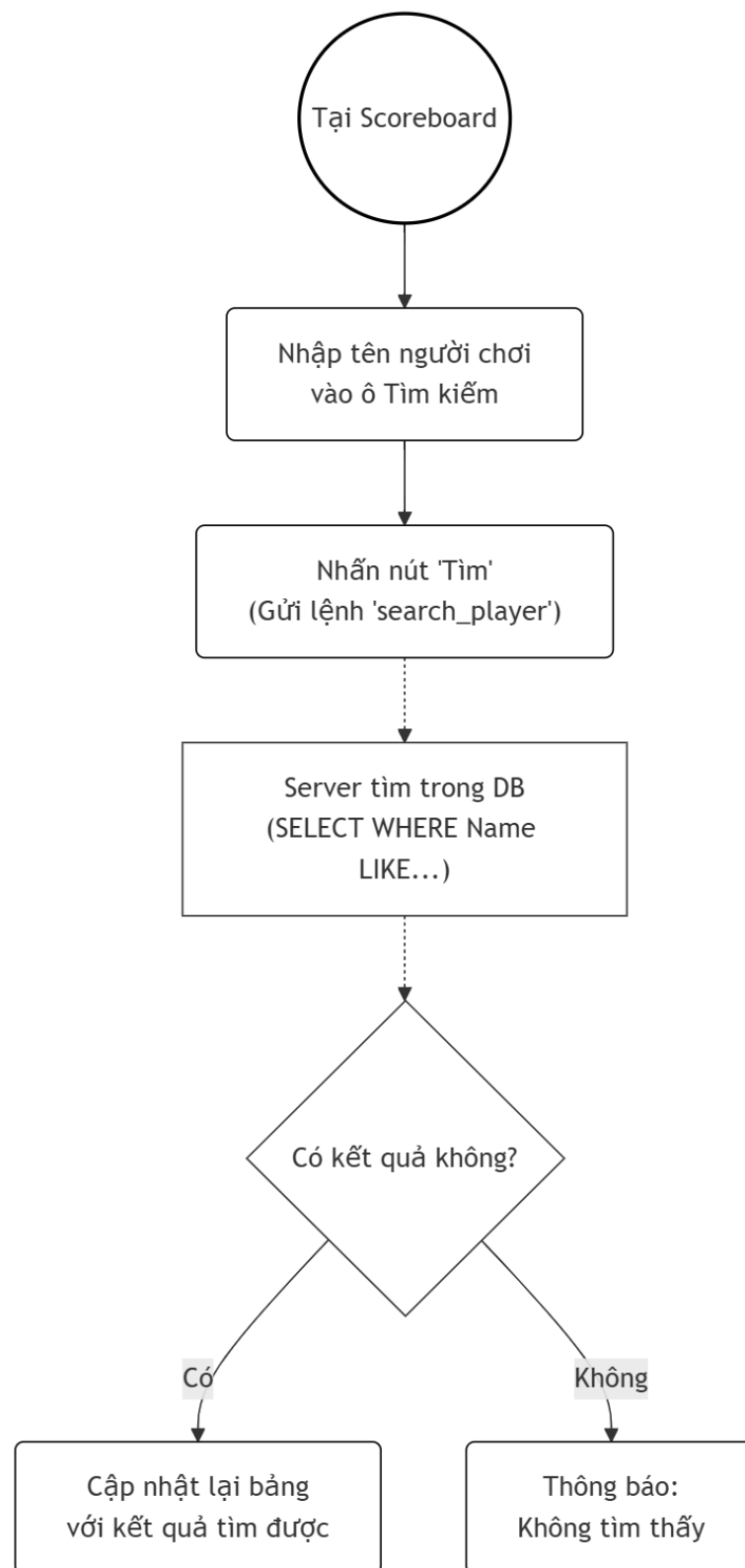
- Vào phòng: Là một người chơi muốn đấu với bạn, tôi muốn nhập ID phòng để tham gia chính xác vào phòng của bạn mình.
 - Flow hoạt động vào phòng:



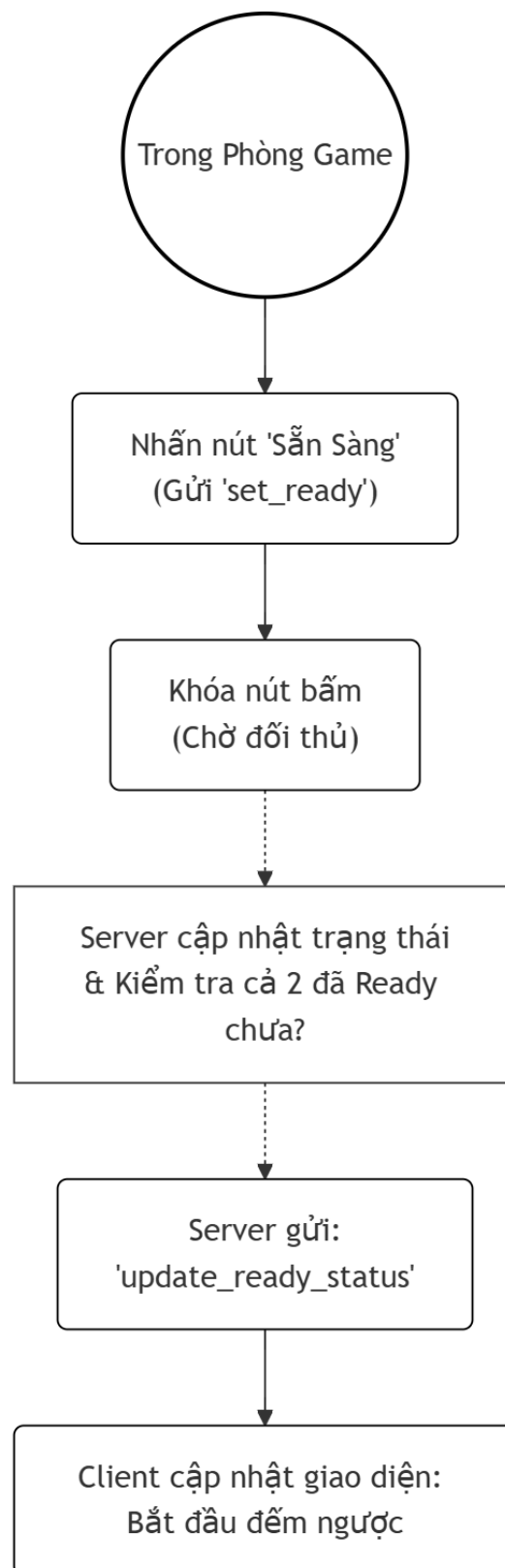
- Xem Bảng xếp hạng: Là một người chơi có tính cạnh tranh, tôi muốn xem danh sách 50 người điểm cao nhất để biết ai là cao thủ trong game.
 - Flow hoạt động xem bảng xếp hạng:



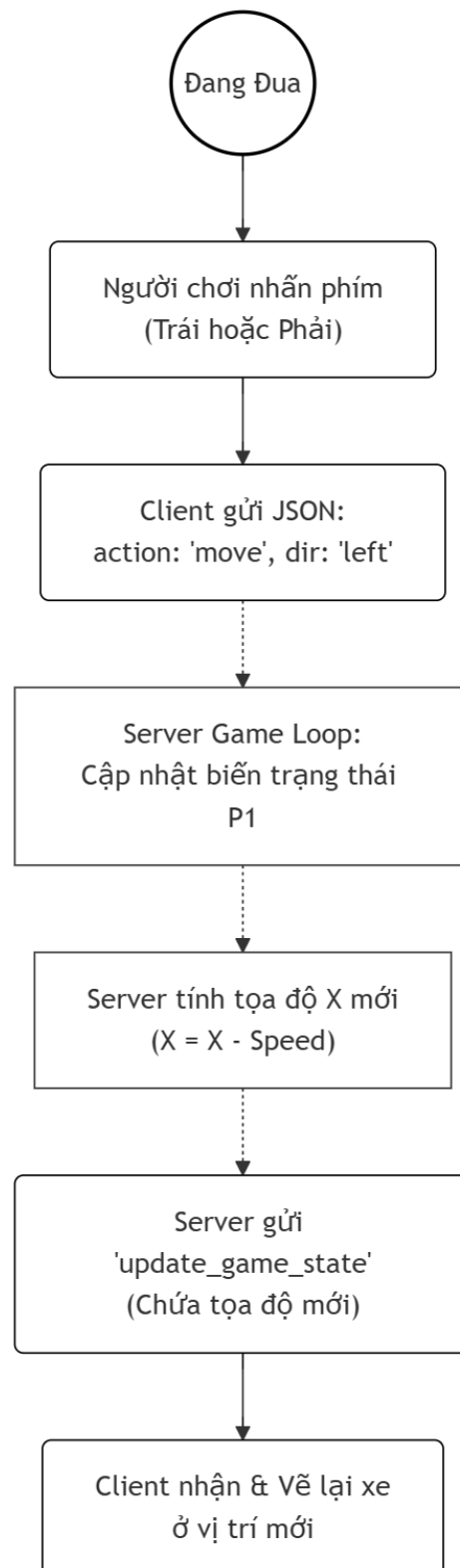
- Tìm kiếm người chơi: Là một người chơi, tôi muốn nhập tên người chơi khác để xem lịch sử thành tích của cụ thể người đó.
 - Flow hoạt động tìm kiếm người chơi:



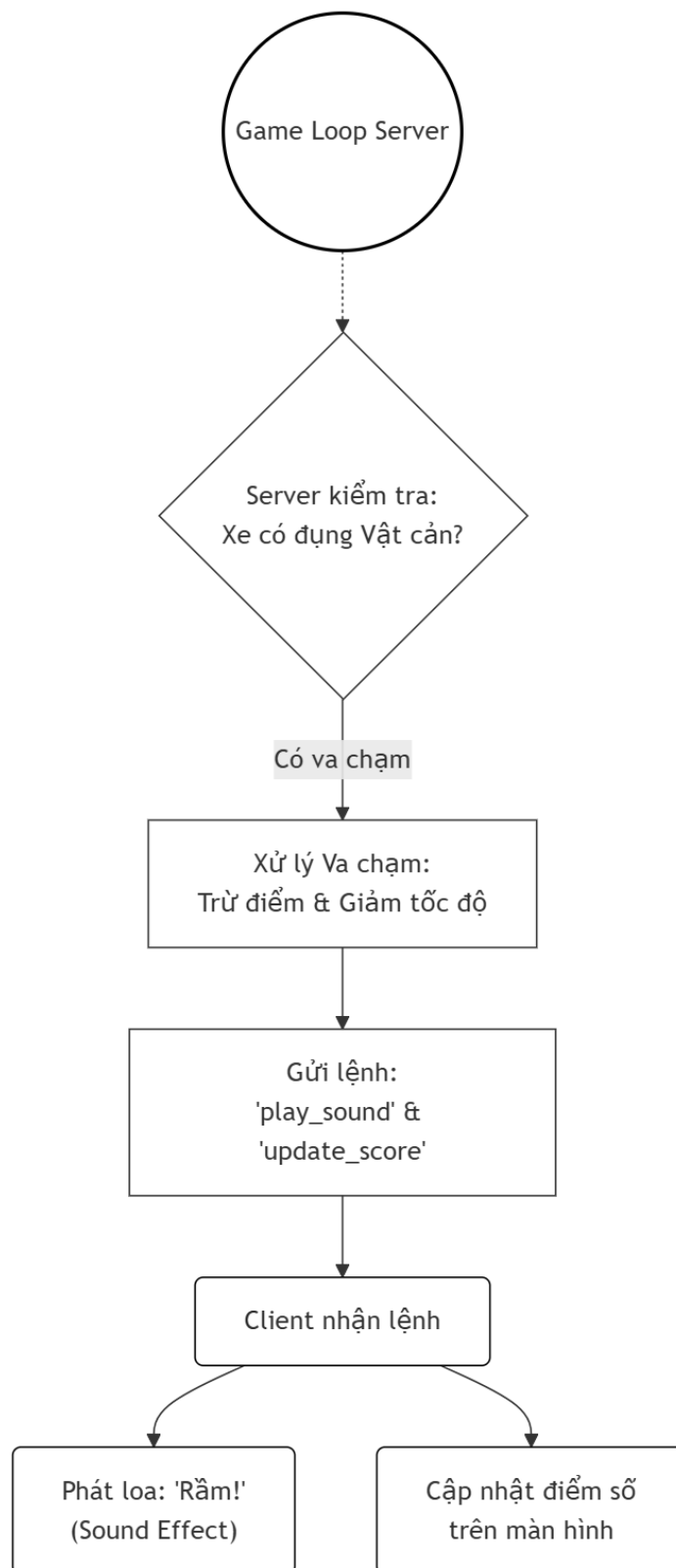
- **Sẵn sàng (Ready):** Là một người chơi trong phòng, tôi muốn bấm nút Sẵn sàng để báo hiệu cho hệ thống biết tôi đã chuẩn bị xong.
 - Flow hoạt động sẵn sàng:



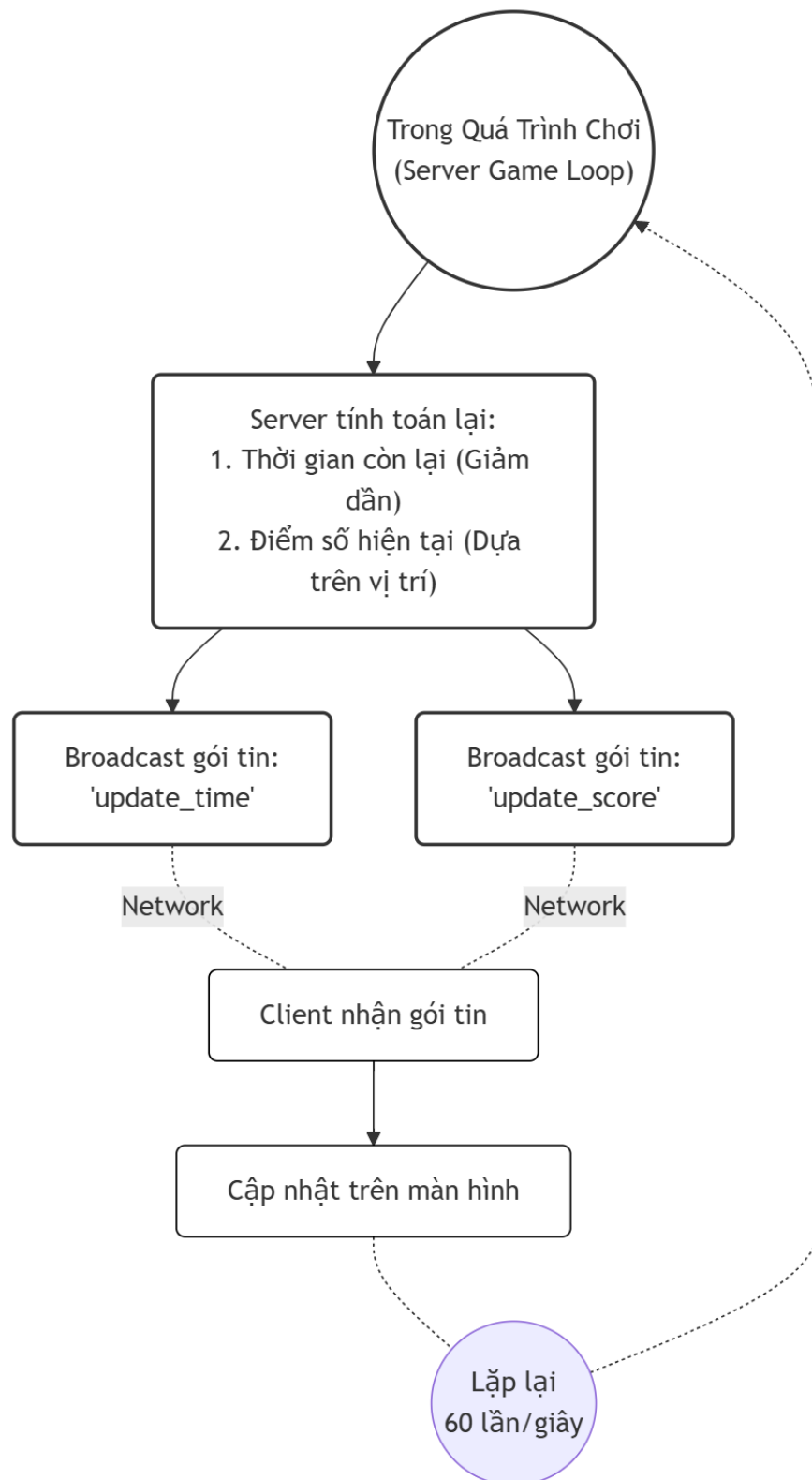
- Điều khiển xe: Là một tay đua, tôi muốn nhấn phím mũi tên Trái/Phải để điều khiển xe tránh các chướng ngại vật trên đường.
 - Flow hoạt động điều khiển xe:



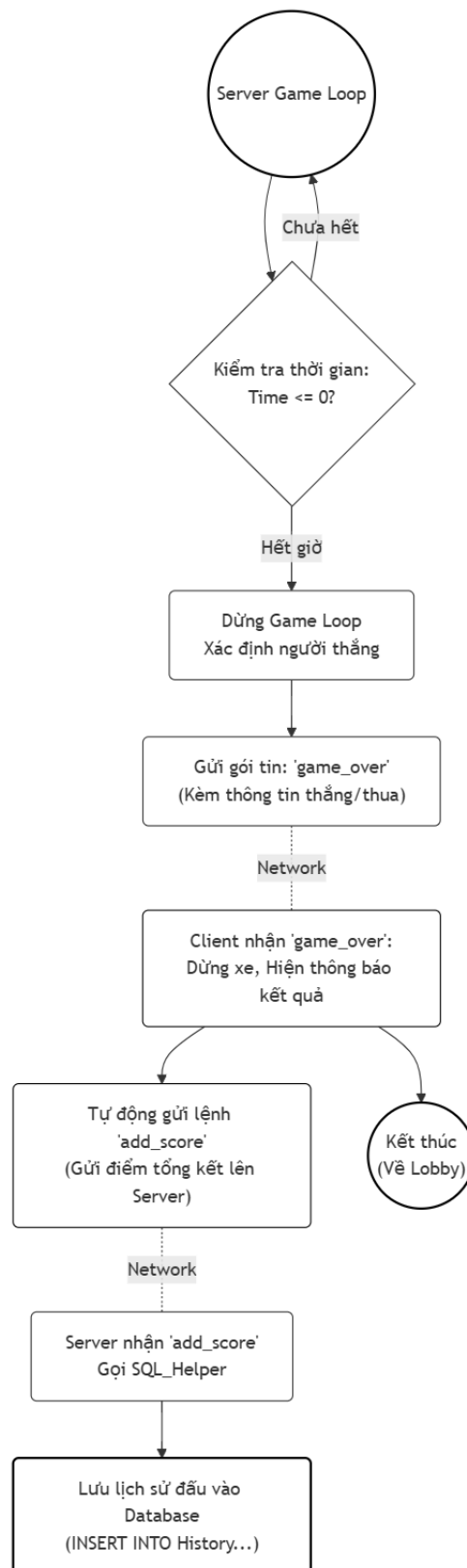
- Nhận phản hồi và chạm: Là một tay đua, tôi muốn nghe thấy âm thanh và thấy xe bị chậm lại khi đụng trúng vật cản để biết mình vừa mắc lỗi.
 - Flow hoạt động nhận phản hồi và chạm:



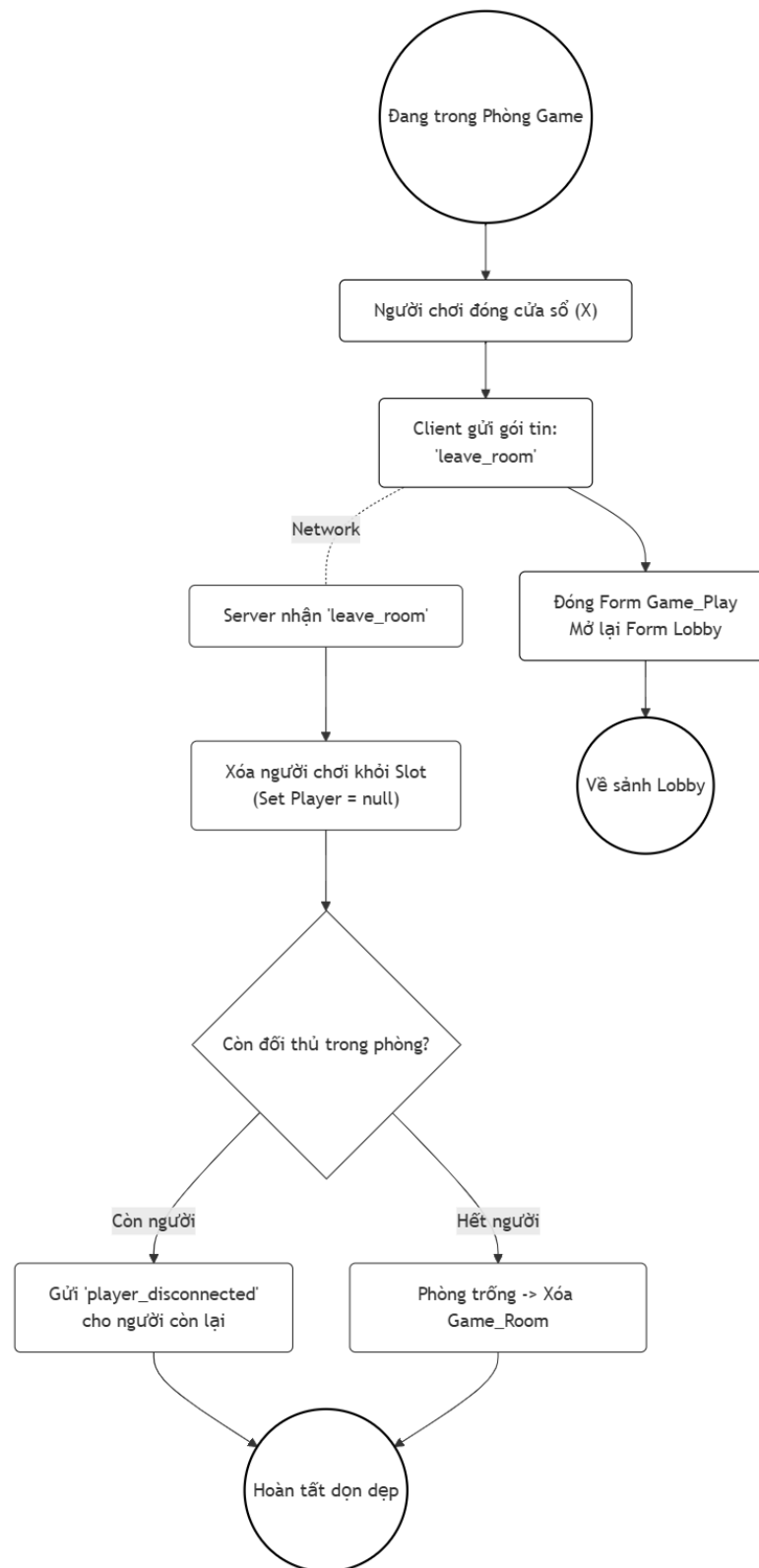
- Xem điểm số/thời gian: Là một tay đua, tôi muốn thấy điểm số và thời gian còn lại nhảy liên tục trên màn hình để biết tình hình trận đấu.
 - Flow hoạt động xem điểm số/thời gian;



- Kết thúc game: Là một người chơi, tôi muốn nhận thông báo khi hết giờ và hệ thống tự động lưu điểm của tôi để tích lũy thành tích.
 - Flow hoạt động kết thúc game:



- Thoát phòng: Là một người chơi, tôi muốn rời khỏi phòng hiện tại để quay trở lại sảnh chờ và tìm đối thủ khác.
 - Flow hoạt động thoát phòng:



4. Use case

4.1. Tổng quan

Biểu đồ Use Case này mô tả các chức năng tương tác của phần mềm Pixel Drift Client dưới góc nhìn của Người Chơi (Player). Hệ thống được chia làm 3 nhóm chức năng chính bao quanh tác nhân: Tài khoản, Sảnh chờ (Lobby) và Gameplay (Trong trận đấu).

4.2. Tác nhân:

Người Chơi (Player): Là người dùng duy nhất tương tác trực tiếp với Client. Người chơi có thể đóng vai trò là Chủ phòng (Host) hoặc Khách (Guest), nhưng về mặt chức năng hệ thống, họ đều có quyền hạn sử dụng các tính năng như nhau.

4.3. Chi tiết các nhóm chức năng

A. Nhóm Quản lý Tài khoản

- **Đăng Ký / Đăng Nhập:** Chức năng cơ bản để xác thực người dùng vào hệ thống.
- **Xem Thông Tin:** Người chơi xem lại hồ sơ cá nhân (ngày sinh, email).
- **Quên Mật Khẩu (Extend):** Đây là chức năng mở rộng của Đăng Nhập. Mỗi quan hệ <<extend>> thể hiện rằng chức năng này chỉ được kích hoạt trong tình huống cụ thể (người dùng quên pass), không phải lúc nào cũng xảy ra.

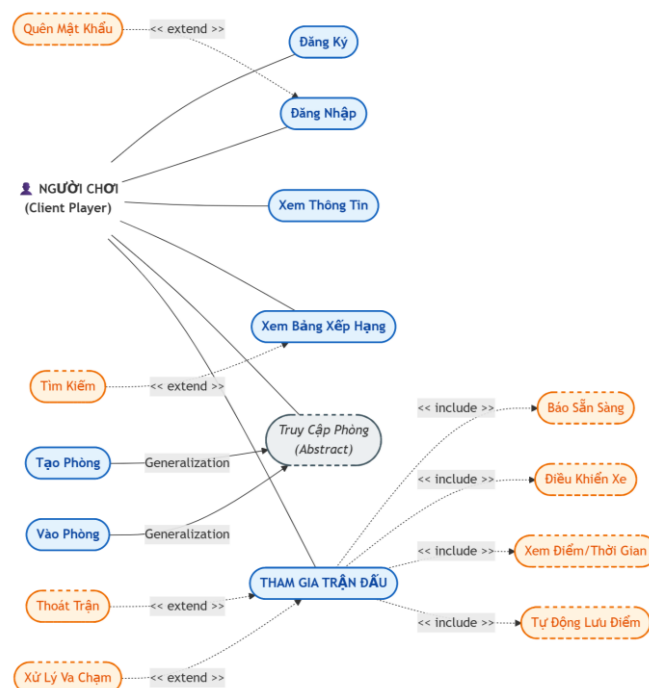
B. Nhóm Sảnh & Bảng Xếp Hạng

- **Xem Bảng Xếp Hạng (BXH):** Hiện thị Top 50 người chơi xuất sắc nhất.
- **Tìm Kiếm (Extend):** Mở rộng cho chức năng Xem BXH. Người dùng có thể chọn lọc danh sách theo tên cụ thể thay vì xem toàn bộ.
- **Truy Cập Phòng (Generalization/Abstract):** Đây là Use Case trừu tượng, đại diện cho hành động "vào game".
 - **Tạo Phòng (Create):** Kế thừa từ Truy Cập Phòng (Người chơi tự tạo ID mới).
 - **Vào Phòng (Join):** Kế thừa từ Truy Cập Phòng (Người chơi nhập ID có sẵn).
 - *Mối quan hệ này thể hiện sự "Tổng quát hóa" (Generalization) - Cả hai hành động đều có chung mục đích là đưa người chơi vào màn hình chờ.*

C. Nhóm Gameplay (Quan trọng nhất) Đây là nhóm chức năng phức tạp nhất, sử dụng nhiều quan hệ ràng buộc:

- **Chức năng chính:** Tham gia trận đấu.
- **Các quan hệ Bao gồm (<<include>>):**
 - **Báo Sẵn Sàng:** Bắt buộc phải thực hiện để game bắt đầu.
 - **Điều Khiển Xe:** Là hành động cốt lõi, không thể tách rời khỏi quá trình chơi.
 - **Xem Điểm/Thời Gian:** Hệ thống bắt buộc phải hiển thị thông số này liên tục.
 - **Tự Động Lưu Điểm:** Khi trận đấu kết thúc, quy trình lưu điểm là bắt buộc và tự động, là một phần không thể thiếu của Use Case chính.
- **Các quan hệ Mở rộng (<<extend>>):**
 - **Thoát Trận:** Người chơi có thể chọn thoát sớm hoặc không (Tùy chọn).
 - **Xử Lý Va Chạm:** Sự kiện này chỉ xảy ra khi xe đâm vào tường hoặc chướng ngại vật (Sự kiện có điều kiện), kích hoạt rung hoặc âm thanh.

4.4 Sơ đồ use case:



5. Database

Info_User			
INTEGER	Id	PK	Primary Key (Tự tăng)
TEXT	Username		Tên đăng nhập (Unique)
TEXT	Email		Email (Unique)
TEXT	Password		Mật khẩu
TEXT	Birthday		Ngày sinh

ScoreBoard			
INTEGER	Id	PK	Primary Key (Tự tăng)
TEXT	PlayerName		Tên người chơi (Lưu dạng text)
INTEGER	WinCount		Số thắng
INTEGER	CrashCount		Số va chạm
REAL	TotalScore		Tổng điểm
DATETIME	DatePlayed		Ngày giờ

5.1. Tổng quan

- Hệ quản trị cơ sở dữ liệu: SQLite.
- Tên file dữ liệu: Qly_Nguoi_Dung.db.
- Cấu trúc: Gồm 2 bảng dữ liệu độc lập là Info_User và ScoreBoard.

5.2. Chi tiết các bảng (Tables)

Bảng 1: Info_User (Thông tin người dùng)

Bảng này dùng để lưu trữ thông tin đăng ký và đăng nhập của tài khoản.

Tên trường (Column)	Kiểu dữ liệu (Type)	Ràng buộc (Constraint)	Mô tả
Id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Mã định danh duy nhất của người dùng, tự động tăng.
Username	TEXT	UNIQUE	Tên đăng nhập của người chơi, không được phép trùng lặp.
Email	TEXT	UNIQUE	Địa chỉ email, dùng để khôi phục tài khoản, không được phép trùng lặp.
Password	TEXT	(None)	Mật khẩu đăng nhập (thường lưu dưới dạng mã hóa).
Birthday	TEXT	(None)	Ngày sinh của người chơi.

Bảng 2: ScoreBoard (Bảng xếp hạng và Lịch sử đấu)

Bảng này lưu trữ chi tiết kết quả của từng trận đấu mà người chơi tham gia. Mỗi khi chơi xong, một dòng mới sẽ được thêm vào bảng này.

Tên trường (Column)	Kiểu dữ liệu (Type)	Ràng buộc (Constraint)	Mô tả
Id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Mã định danh duy nhất của lượt chơi, tự động tăng.
PlayerName	TEXT	NOT NULL	Tên người chơi tham gia trận đấu (được dùng để liên kết logic với bảng Info_User).
WinCount	INTEGER	DEFAULT 0	Số trận thắng trong lượt chơi đó.
CrashCount	INTEGER	DEFAULT 0	Số lần va chạm (chỉ số phụ để xếp hạng).
TotalScore	REAL	DEFAULT 0	Tổng điểm đạt được (kiểu số thực).
DatePlayed	DATETIME	DEFAULT CURRENT_TIMESTAMP	Thời gian trận đấu diễn ra, mặc định lấy giờ hệ thống hiện tại.

5.3. Mối quan hệ giữa các bảng (Relationship Analysis)

- Về mặt vật lý (Physical): Hai bảng này độc lập hoàn toàn, không thiết lập Khóa Ngoại (FOREIGN KEY) trong câu lệnh SQL CREATE TABLE.
- Về mặt logic (Logical): Hệ thống quản lý mối quan hệ thông qua trường Username (bảng Info_User) và PlayerName (bảng ScoreBoard).

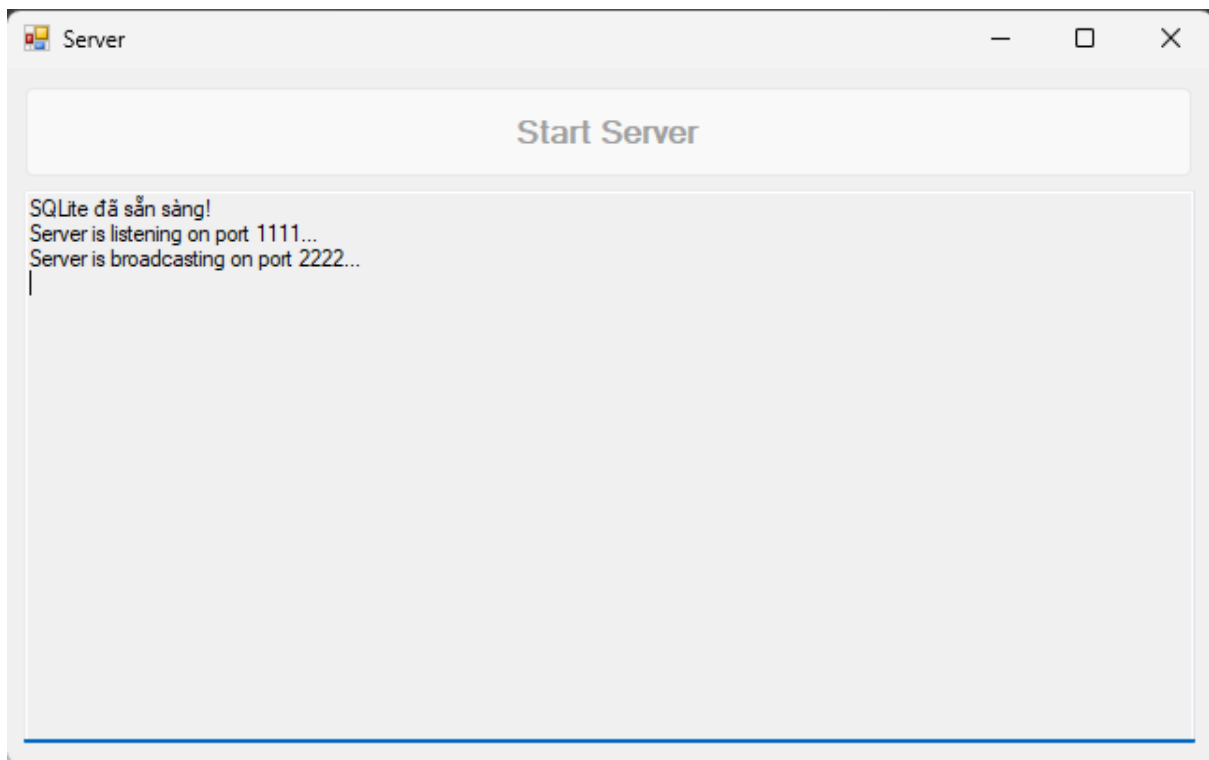
- Khi truy vấn Bảng xếp hạng (GetTopScores), hệ thống sẽ gom nhóm (GROUP BY) theo PlayerName để tính tổng thành tích của một người chơi.
- Mỗi quan hệ là 1 - n (Một - Nhiều): Một tài khoản (Info_User) có thể có nhiều dòng lịch sử thi đấu (ScoreBoard).

Chương IV: XÂY DỰNG ỨNG DỤNG

1. Tương quan giữa mục tiêu lý thuyết và thực tế

- Phần mềm ứng dụng đáp ứng được các mục tiêu đã đề ra ở mục tiêu lý thuyết.

2. Các giao diện chính của ứng dụng

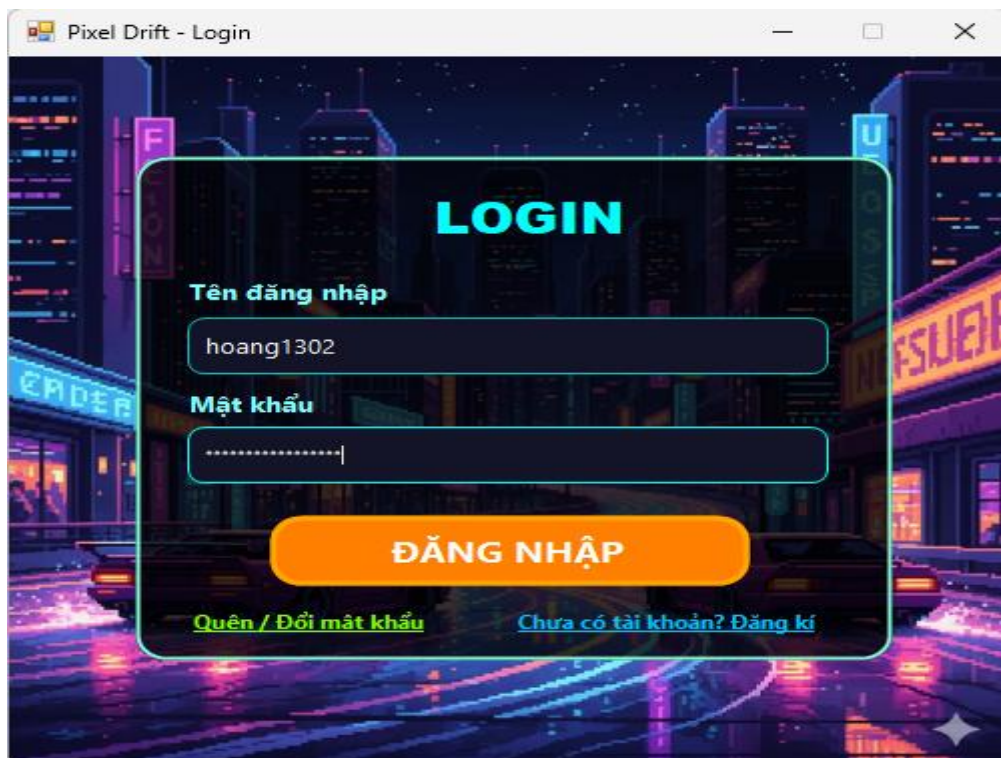


Hình 1 Giao diện Server

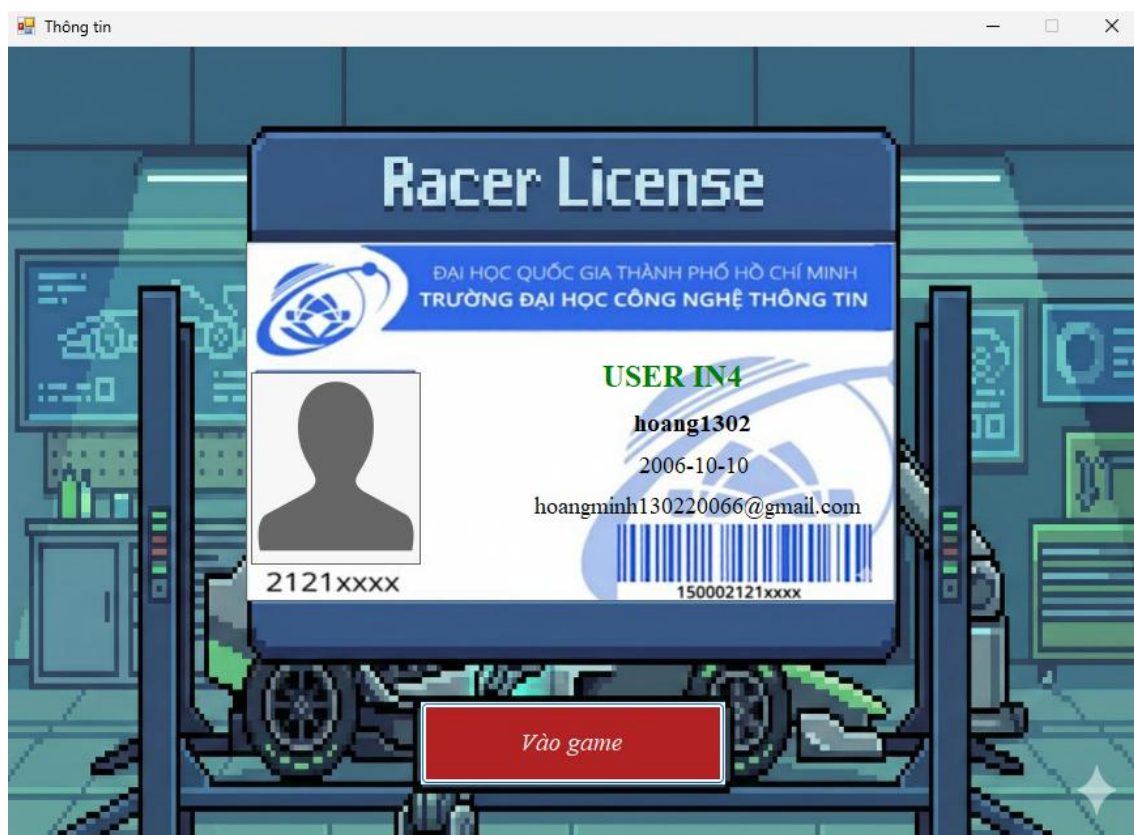


Hình 2 Giao diện Mở đầu

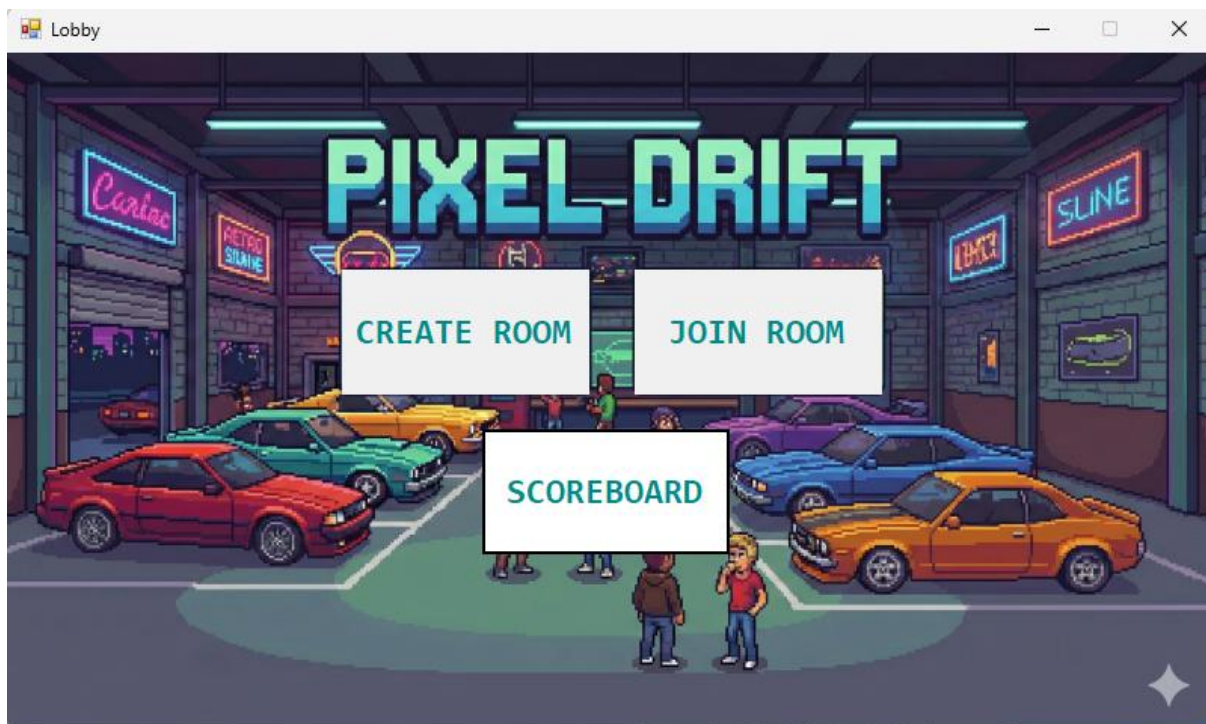
Hình 3 Giao diện Đăng ký



Hình 4 Giao diện Đăng nhập



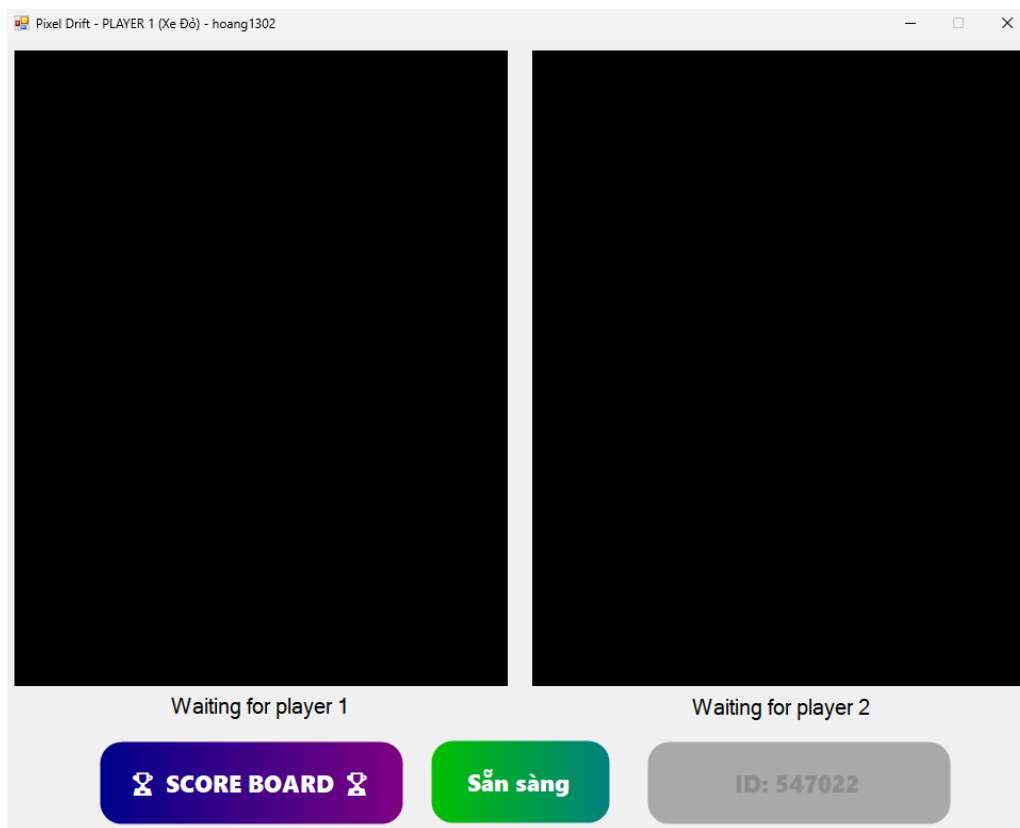
Hình 5 Giao diện Thông tin người chơi



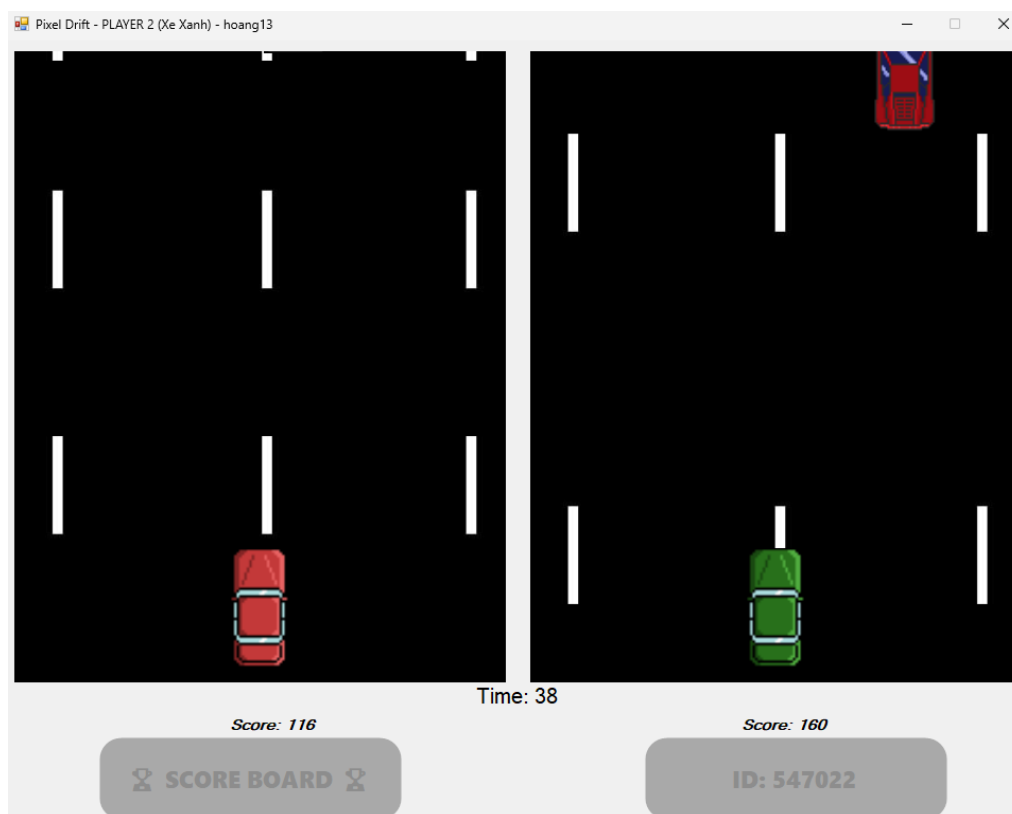
Hình 6 Giao diện Lobby (Tạo phòng/Vào phòng/Xem bảng điểm)



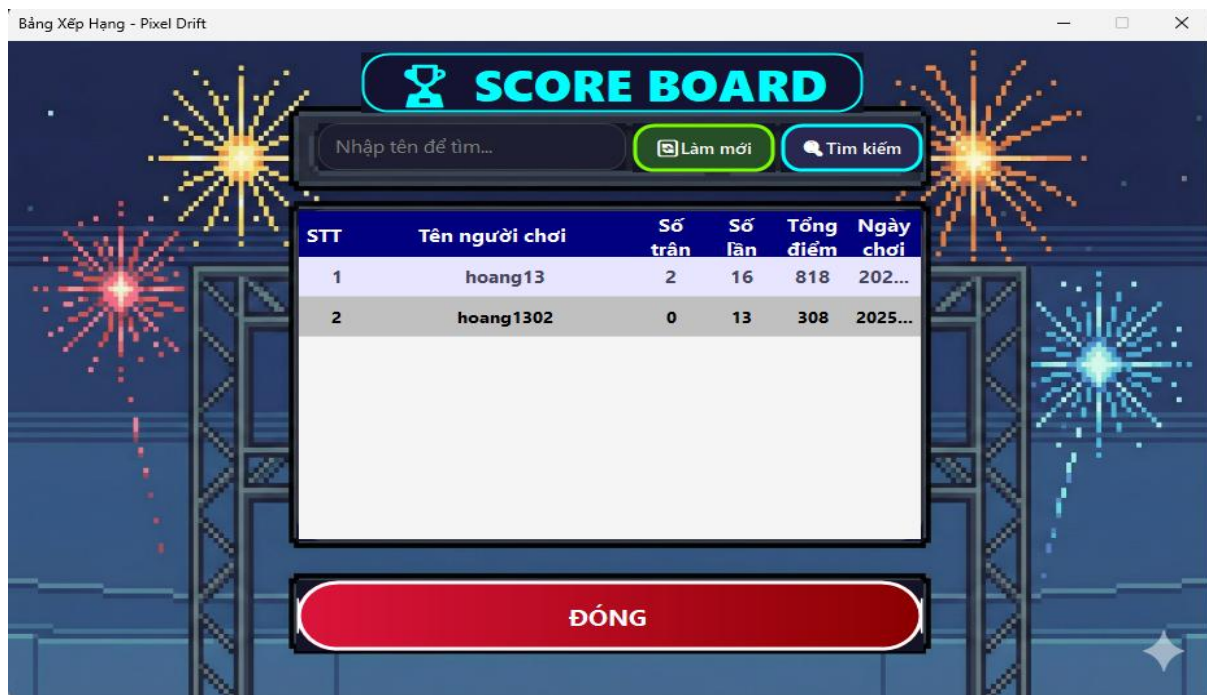
Hình 7 Giao diện Nhập ID (Tìm phòng)



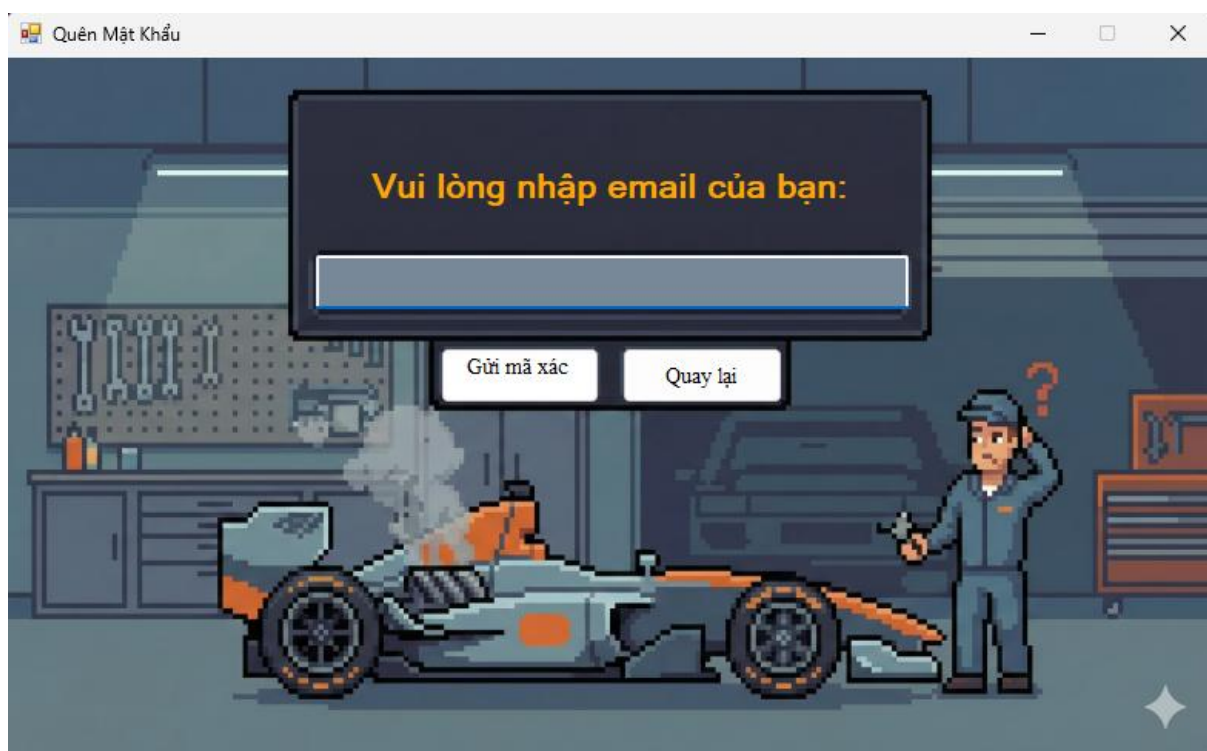
Hình 8 Giao diện Phòng game (Trước khi bắt đầu)



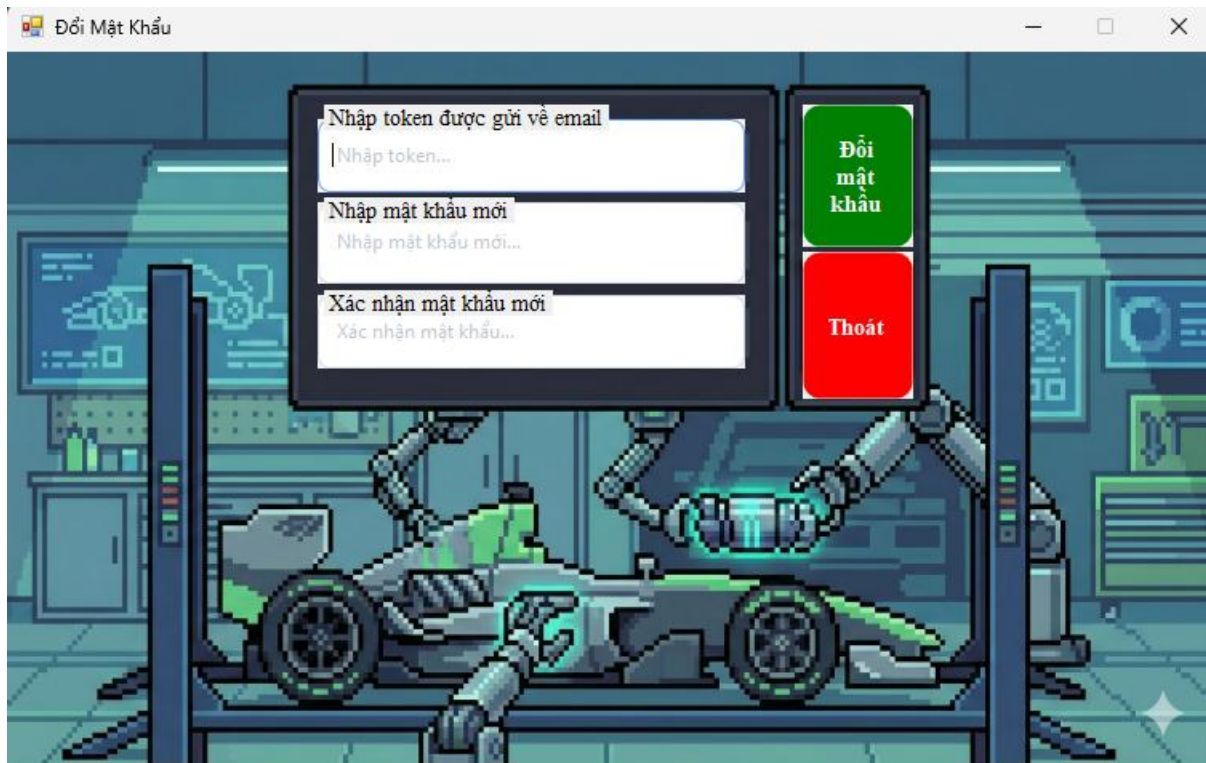
Hình 9 Giao diện Phòng game (Khi bắt đầu)



Hình 10 Giao diện Bảng xếp hạng



Hình 11 Giao diện Quên mật khẩu



Hình 12 Giao diện Đổi mật khẩu

3. Kết quả đề tài

Sau quá trình nghiên cứu và thực hiện, đồ án xây dựng game "Pixel Drift" đã hoàn thành và đáp ứng tốt các yêu cầu đặt ra ban đầu, cụ thể như sau:

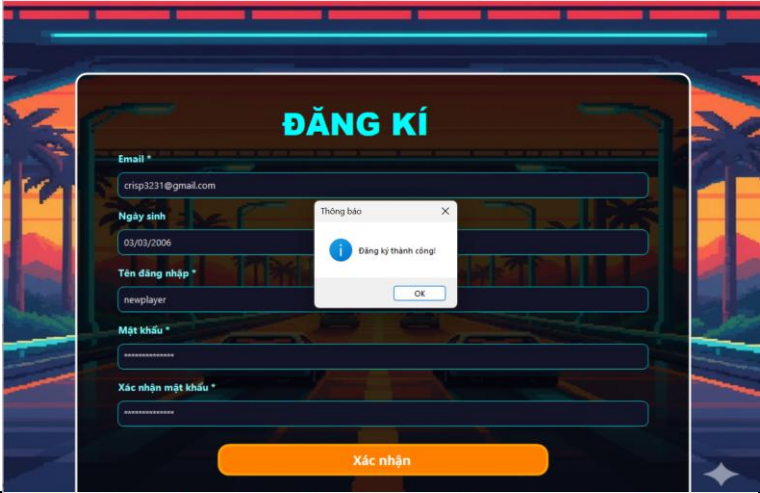
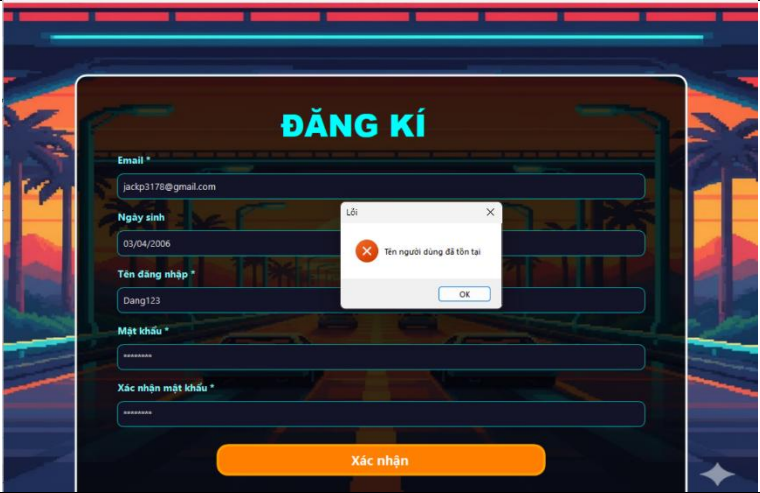
- **Về Trải nghiệm Người dùng và Giao diện (UI):**
 - Ứng dụng đã xây dựng được một hệ thống giao diện WinForms hoàn chỉnh, trực quan và thân thiện với người dùng.
 - Quy trình trải nghiệm được thiết kế liền mạch từ khâu **Đăng nhập/Đăng ký**, chuyển tiếp sang **Sảnh chờ (Lobby)** để xem bảng xếp hạng, và cuối cùng là màn hình **Game Play** với các hiệu ứng hình ảnh rõ ràng.
 - Việc cài đặt và triển khai ứng dụng đơn giản, hoạt động ổn định trên môi trường Windows và tương thích tốt với các mạng ảo như Radmin VPN để chơi qua mạng LAN ảo.
- **Về Hiệu năng Kỹ thuật và Kết nối Mạng (Networking):**
 - Đồ án đã giải quyết thành công bài toán **đồng bộ dữ liệu thời gian thực (Real-time)** trong game tốc độ cao.
 - Thay vì sử dụng các phương thức truyền thống gây độ trễ cao như ghi đọc liên tục vào Database, nhóm đã áp dụng thành công kỹ thuật **lập trình mạng TCP Socket**.

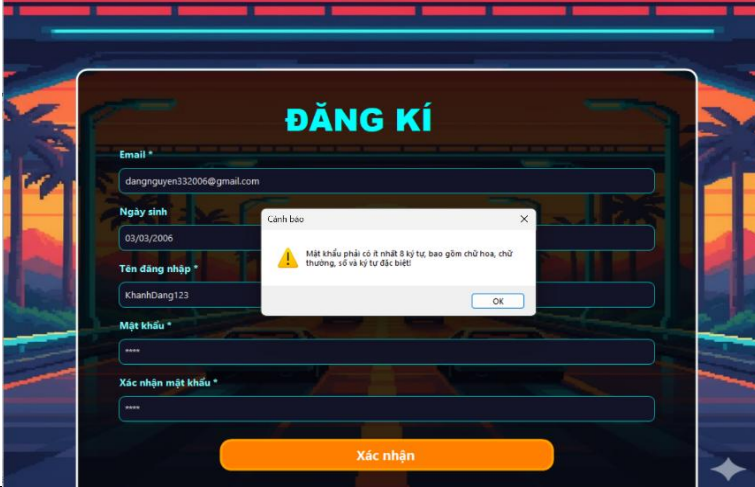
- Cơ chế truyền tải gói tin JSON chứa tọa độ và trạng thái xe được thực hiện liên tục (với tần suất cao), đảm bảo chuyển động của đối thủ trên màn hình luôn mượt mà, đồng thời giảm thiểu tối đa độ trễ (latency) giữa hai máy Client.
- **Về Quản lý và Lưu trữ Dữ liệu (Database):**
 - Hệ thống đã tích hợp thành công cơ sở dữ liệu **SQLite** để đảm bảo tính bền vững của dữ liệu.
 - Cấu trúc cơ sở dữ liệu được thiết kế tối ưu với hai bảng riêng biệt:
 - **Bảng Info_User:** Quản lý chặt chẽ thông tin định danh và bảo mật tài khoản người dùng.
 - **Bảng ScoreBoard:** Lưu trữ lịch sử thi đấu và thành tích, phục vụ cho tính năng Bảng xếp hạng và Thống kê.
 - Quy trình xử lý dữ liệu được phân tách rõ ràng: Chỉ ghi xuống Database khi kết thúc ván chơi, giúp giảm tải cho hệ thống trong quá trình thi đấu.

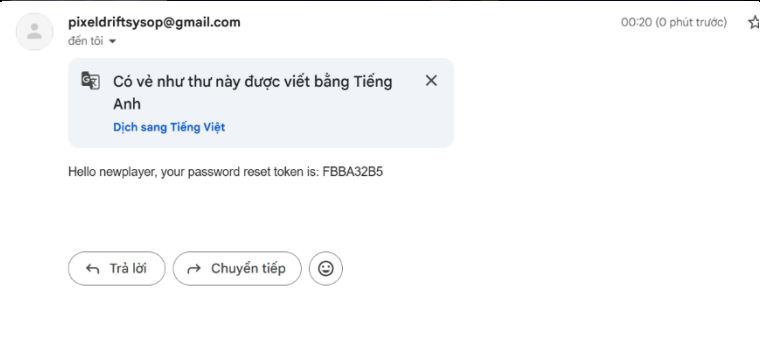
Kết luận chung: Đồ án không chỉ tạo ra một sản phẩm game giải trí hoàn thiện mà còn chứng minh được khả năng áp dụng các kiến thức nền tảng về Lập trình mạng, Cơ sở dữ liệu và Kiến trúc Client-Server vào thực tế. Hệ thống hoạt động ổn định, đáp ứng tốt nhu cầu chơi game đối kháng giữa hai người chơi trong cùng một mạng

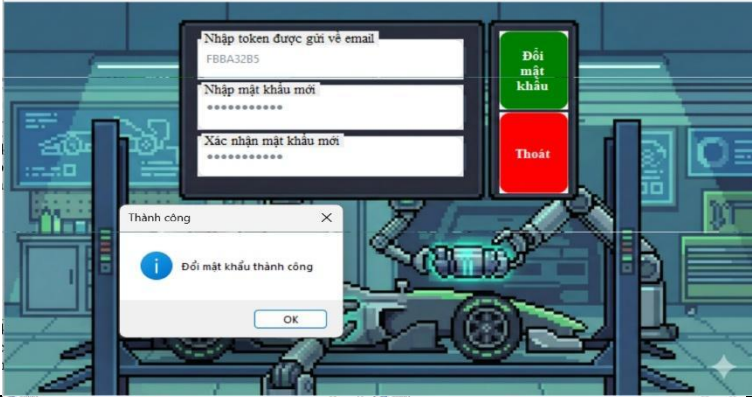
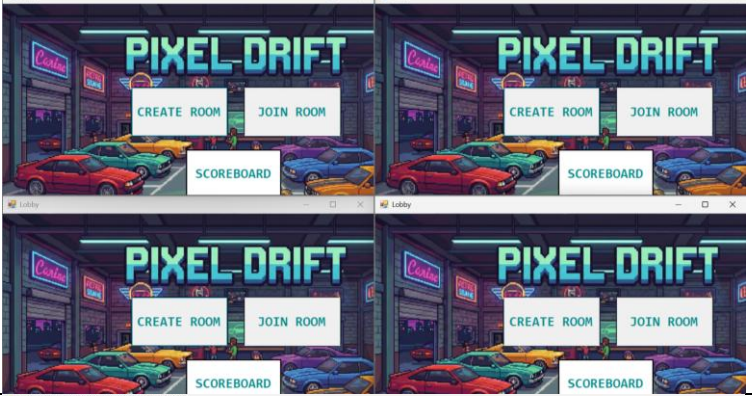
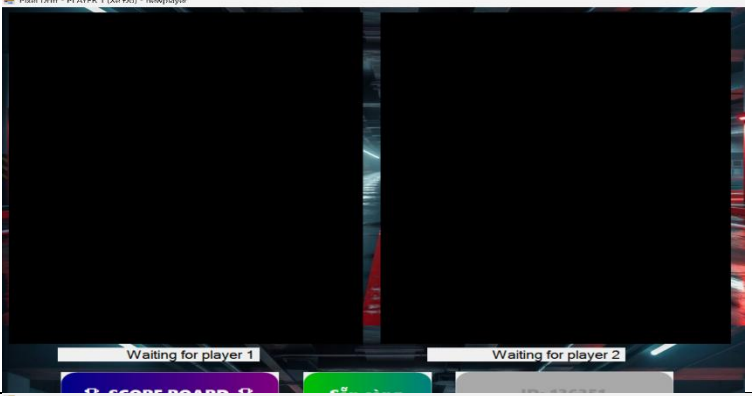
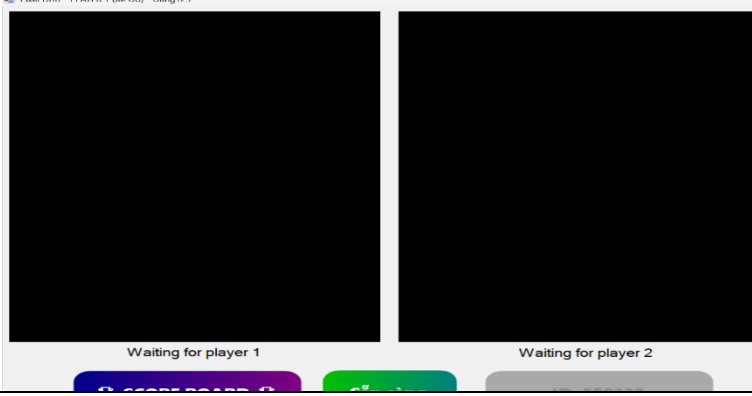
Chương V: KIỂM TRA ỨNG DỤNG

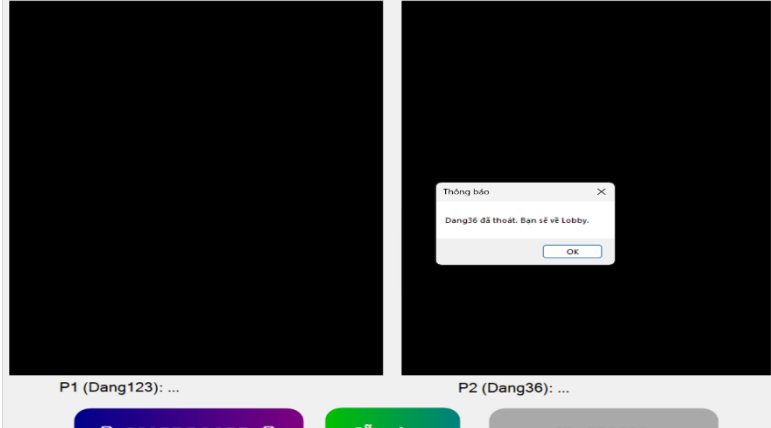
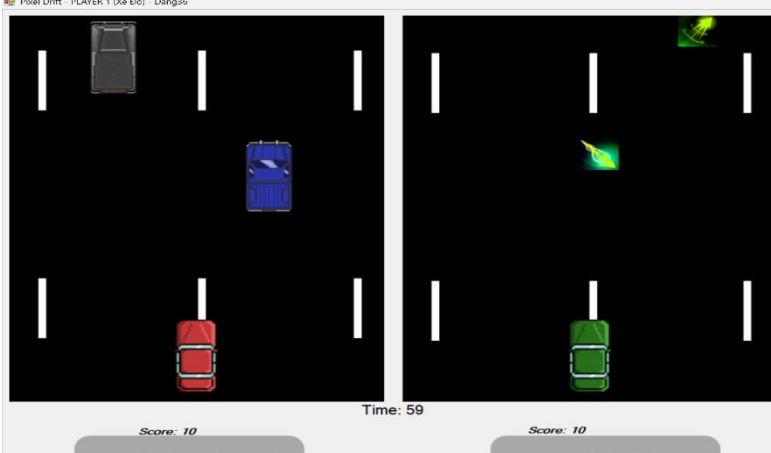
1. Kiểm tra chức năng sản phẩm

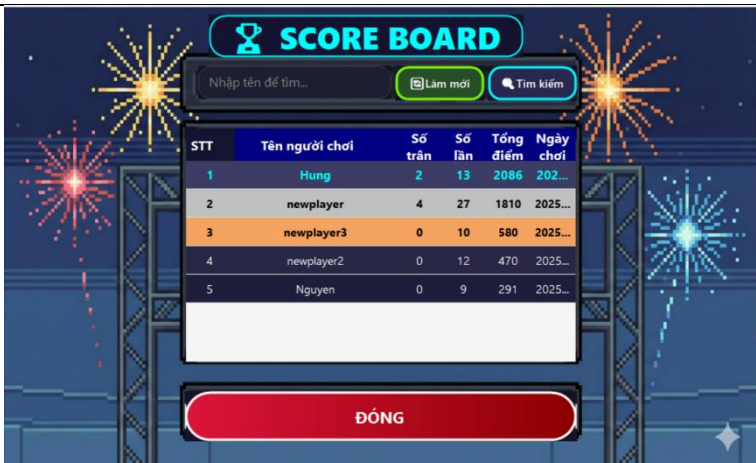
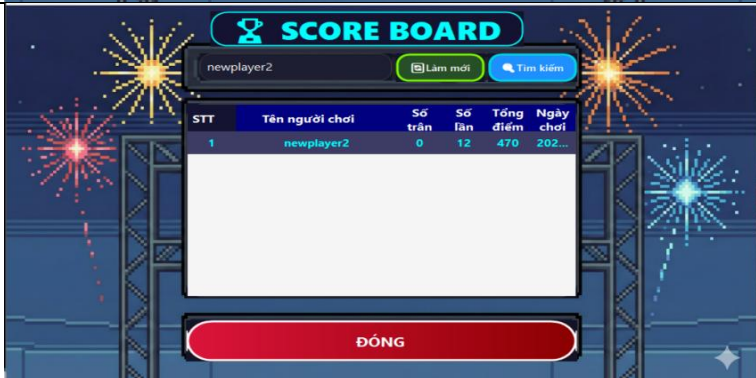
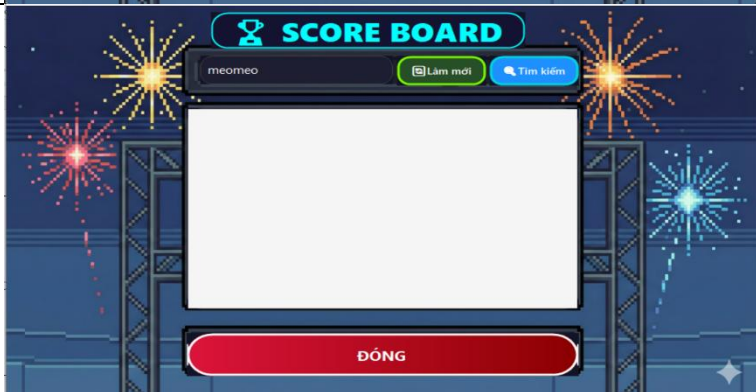
<i>Mã testcase</i>	<i>Mô tả</i>	<i>Hình ảnh</i>	<i>Kết quả</i>
TC001	Đăng ký một tài khoản		Thông báo tạo tài khoản thành công
TC002	(validation) đăng ký trùng tên đăng nhập		Thông báo đăng ký thất bại vì cùng tên người dùng đã tồn tại

TC003	(validation) Đăng ký trùng tài khoản gmail		Thông báo đăng kí thất bại vì email đã tồn tại một tài khoản
TC004	(validation) Đăng ký mật khẩu không hợp lệ		Thông báo mật khẩu không hợp lệ
TC005	Đăng nhập vào game		Thông báo đăng nhập game thành công

TC006	(validation) Đăng nhập vào một tài khoản chưa tồn tại		Thông báo tài khoản không tồn tại
TC007	(validation) Không đúng tên đăng nhập / mật khẩu		Thông báo đăng nhập sai tài khoản / mật khẩu
TC008	(validation) Hai tài khoản cùng đăng nhập		Thông báo với người đăng nhập vào tài khoản trước là tài khoản đang được sử dụng và logout tài khoản
TC009	Quên mật khẩu		Nhận được mail token xác nhận đổi mật khẩu

TC010	Đổi mật khẩu mới		Đổi mật khẩu thành công
TC011	Nhiều người dùng đăng nhập cùng lúc		Server quản lý nhiều client
TC012	Tạo phòng chơi mới		Phòng chơi mới được tạo ra
TC013	Người dùng tham gia vào phòng chơi		Vào phòng chơi thành công

TC014	(validation) Người dùng đăng nhập vào một phòng chơi đã có đủ người		Một thông báo phòng đã đủ người và người chơi không thể vào phòng
TC015	(validation) Người dùng đăng nhập vào một phòng chơi không tồn tại		Một thông báo phòng chưa tồn tại và người chơi không thể vào phòng
TC016	(validation) Một người dùng đã vào phòng nhưng sau đó thoát ra		Thông báo đến player còn lại và player đó biến mất khỏi phòng
TC017	Hai player cùng nhau chơi game		Game hoạt động bình thường

TC018	Hiển thị bảng xếp hạng	 <table><thead><tr><th>STT</th><th>Tên người chơi</th><th>Số trận</th><th>Số lần</th><th>Tổng điểm</th><th>Ngày chơi</th></tr></thead><tbody><tr><td>1</td><td>Hung</td><td>2</td><td>13</td><td>2086</td><td>202...</td></tr><tr><td>2</td><td>newplayer</td><td>4</td><td>27</td><td>1810</td><td>2025...</td></tr><tr><td>3</td><td>newplayer3</td><td>0</td><td>10</td><td>580</td><td>2025...</td></tr><tr><td>4</td><td>newplayer2</td><td>0</td><td>12</td><td>470</td><td>2025...</td></tr><tr><td>5</td><td>Nguyen</td><td>0</td><td>9</td><td>291</td><td>2025...</td></tr></tbody></table>	STT	Tên người chơi	Số trận	Số lần	Tổng điểm	Ngày chơi	1	Hung	2	13	2086	202...	2	newplayer	4	27	1810	2025...	3	newplayer3	0	10	580	2025...	4	newplayer2	0	12	470	2025...	5	Nguyen	0	9	291	2025...	Hiển thị bảng xếp hạng của người chơi
STT	Tên người chơi	Số trận	Số lần	Tổng điểm	Ngày chơi																																		
1	Hung	2	13	2086	202...																																		
2	newplayer	4	27	1810	2025...																																		
3	newplayer3	0	10	580	2025...																																		
4	newplayer2	0	12	470	2025...																																		
5	Nguyen	0	9	291	2025...																																		
TC019	Tìm kiếm người dùng theo tên	 <table><thead><tr><th>STT</th><th>Tên người chơi</th><th>Số trận</th><th>Số lần</th><th>Tổng điểm</th><th>Ngày chơi</th></tr></thead><tbody><tr><td>1</td><td>newplayer2</td><td>0</td><td>12</td><td>470</td><td>202...</td></tr></tbody></table>	STT	Tên người chơi	Số trận	Số lần	Tổng điểm	Ngày chơi	1	newplayer2	0	12	470	202...	Chỉ hiển thị thông tin người đó																								
STT	Tên người chơi	Số trận	Số lần	Tổng điểm	Ngày chơi																																		
1	newplayer2	0	12	470	202...																																		
TC020	(validation) Tìm kiếm một người dùng không tồn tại	 <table><thead><tr><th>STT</th><th>Tên người chơi</th><th>Số trận</th><th>Số lần</th><th>Tổng điểm</th><th>Ngày chơi</th></tr></thead><tbody></tbody></table>	STT	Tên người chơi	Số trận	Số lần	Tổng điểm	Ngày chơi	Không hiển thị kết quả																														
STT	Tên người chơi	Số trận	Số lần	Tổng điểm	Ngày chơi																																		

2. Khả năng chịu tải

- Ứng dụng có chịu tải tối đa 20 người chơi cùng một lúc mà vẫn hoạt động mượt mà

3. Độ trễ

- Thời gian trung bình để đồng bộ dữ liệu với cơ sở dữ liệu SQLite là khoảng 500ms. Các thao tác như đăng nhập, đăng kí và xem thông tin người chơi đều được xử lý nhanh chóng, đảm bảo trải nghiệm người dùng.
- Đối với game room, tốc độ gửi gói tin cập nhật vị trí, số điểm, ... từ server đến client nằm ở khoảng 16,67ms

4. Các tác động môi trường

- Ứng dụng vẫn hoạt động bình thường khi mạng yếu nhưng trong quá trình chơi game dễ bị vỡ khung hình dẫn đến trải nghiệm không tốt khi chơi game.
- Các lỗi hệ thống được xử lý bằng cách thông báo lỗi cụ thể và gợi ý người dùng các bước khắc phục.

5. Kết luận

- Ứng dụng có khả năng hoạt động tương đối ổn định trong các điều kiện môi trường khác nhau

Chương VI: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

1. Kết luận

Qua quá trình phân tích, thiết kế, xây dựng và kiểm thử, ứng dụng "Pixel Driff" đã hoàn thiện và đáp ứng được các yêu cầu đề ra. Ứng dụng giúp mang lại một trải nghiệm giải trí đầy thú vị và hoài niệm đối với các tựa game được phát hành vào đầu những năm 2000.

2. Ưu điểm

- Giao diện được thiết kế dựa trên phong cách đồ họa pixel hiện đại, thân thiện và dễ dàng thao tác
- Ứng dụng hoạt động tốt trong các điều kiện môi trường khác nhau
- Đáp ứng được các yêu cầu cơ bản của một trò chơi điện tử như đăng nhập, đăng ký, scoreboard và quản lý người chơi theo từng phòng.
- Có cơ chế bảo vệ thông tin tài khoản của người dùng
- Hỗ trợ chức năng đổi mật khẩu tài khoản người dùng
- Người chơi có thể chơi ở nhiều địa điểm khác nhau thông qua radmin mà không cần phải ở địa điểm đặt máy chủ
- Đảm bảo tài khoản người dùng chỉ có thể truy cập bằng một thiết bị tại một thời điểm

3. Nhược điểm

- Chưa tối ưu hóa toàn bộ giao diện cho các thiết bị có màn hình nhỏ.
- Cần cải thiện tính năng trao đổi dữ liệu giữa client và server

4. Hướng phát triển

- **Tối ưu hóa giao diện:** Cải thiện giao diện để phù hợp hơn bằng các thiết bị có màn hình nhỏ.
- **Mở rộng nền tảng:** phát hành các phiên bản cho Android và IOS

- ***Phát triển thêm tính năng***: thêm các tính năng cho phép người dùng ghép trận nhanh, hệ thống skin xe, đường đua đa dạng và thêm hệ thống VIP chứa các tính năng độc quyền
- ***Tăng khả năng chịu tải***: từ chỉ có thể hỗ trợ tối đa 20 người lên tới hơn 1000 người chơi hoạt động cùng một lúc

TÀI LIỆU THAM KHẢO

1. BillWagner. (n.d.). *.NET documentation*. Microsoft.com. Retrieved December 11, 2025, from <https://learn.microsoft.com/en-us/dotnet/>
2. Tài liệu hướng dẫn thực hành Lập trình mạng căn bản - Tác giả: Đỗ Thị Hương Lan, Trần Hồng Nghi.
3. *SQLite Documentation*. (n.d.). Sqlite.org. Retrieved December 11, 2025, from <https://sqlite.org/docs.html>